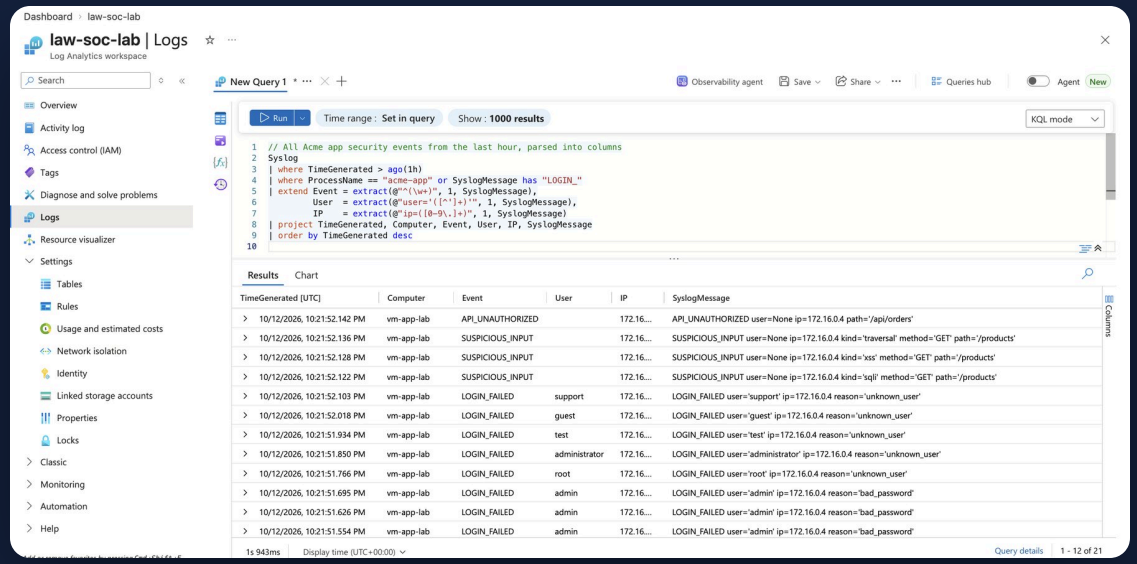
CLOUD SECURITY REFERENCE BUILD · AZURE

Private AI for the SOC on Azure

Keyless Azure OpenAI over Private Link, feeding Microsoft Sentinel

GabriSec · October 2026



INTRODUCTION

At a glance

THE PROBLEM

A SOC wants a language model to triage security events, but those events are among the most sensitive data a company holds, and an AI endpoint is a new attack surface: one more API that can be reached and one more key that can leak.

WHAT WAS BUILT

Azure OpenAI with no public path and no API keys, called by managed identities that hold an inference-only role. A hub-and-spoke network with Private Link, private DNS, NAT-only outbound and Azure Bastion with Entra ID login. A log pipeline from a realistic web application through the Azure Monitor Agent into Log Analytics and Microsoft Sentinel, under Azure Policy guardrails, group-based RBAC and cost limits.

INTRODUCTION

Why this project

Security teams want to use generative AI on their security data, but that data is among the most sensitive they own. It must not travel across the public internet, and a single API key forgotten in a script must never be enough to use the model.

This project builds the foundation of a Security Operations Center on Microsoft Azure around one question: **how can a SOC use a language model on its most sensitive data without creating a new attack surface?**

The AI service has no public path and no API keys, every caller proves its identity through Microsoft Entra ID instead of a stored secret, each identity receives only the permissions it needs, security logs are centralized, and costs have hard limits from day one.

The document shows the design and implementation work behind a secure AI and SIEM environment, and it is a complete, reproducible tutorial: every phase is described step by step, explains why it exists and how its result is used later, and ends with a test that proves the control works. The screenshots come from the build in Italy North in October 2026.

INTRODUCTION

How to read this guide

Many values depend on your organization, your subscription and your use case, so every configuration value carries one of two markers. In the tables, a **Type** column says which one applies to each value. A value marked Customizable is one you choose according to your requirements, such as the region, the availability zone, the VM size, the disk type, the resource names, the address ranges or the budget limits. A value marked Required must stay as shown, because the architecture, a security control or a later step depends on it.

Placeholders in commands are written in angle brackets, for example `<RESOURCE_GROUP>` or `<APP_PRIVATE_IP>`, and are replaced with your own values. Azure CLI commands run in Azure Cloud Shell with Bash unless the slide says otherwise, and commands that run on a virtual machine name the machine they belong to.

The environment is built in two stages. During the build phase, Parts 3 to 9, Azure Bastion and Entra ID login do not exist yet, so the jump host is reached with an SSH key through a public IP restricted to your own address, as temporary bootstrap access. Part 10 then moves administration to the final state, Bastion with Entra ID authentication, and removes the public IP, the SSH rule and the key.

The names used here are the ones of this project. In an organization they follow the company naming convention, typically a pattern such as `<type>-<workload>-<environment>-<region>-<instance>`, which produces names like `rg-soc-prod-itn-001`.

INTRODUCTION

What is in place at the end

The resource group is governed by Azure Policy guardrails and ownership tags, so every component of the project is evaluated against the organization's rules for region, ownership, AI network access and AI key access.

At the center sits a private, keyless AI service. Azure OpenAI has its API keys disabled and its public network access disabled, so it can be reached only through a private endpoint, by callers that present a Microsoft Entra ID token.

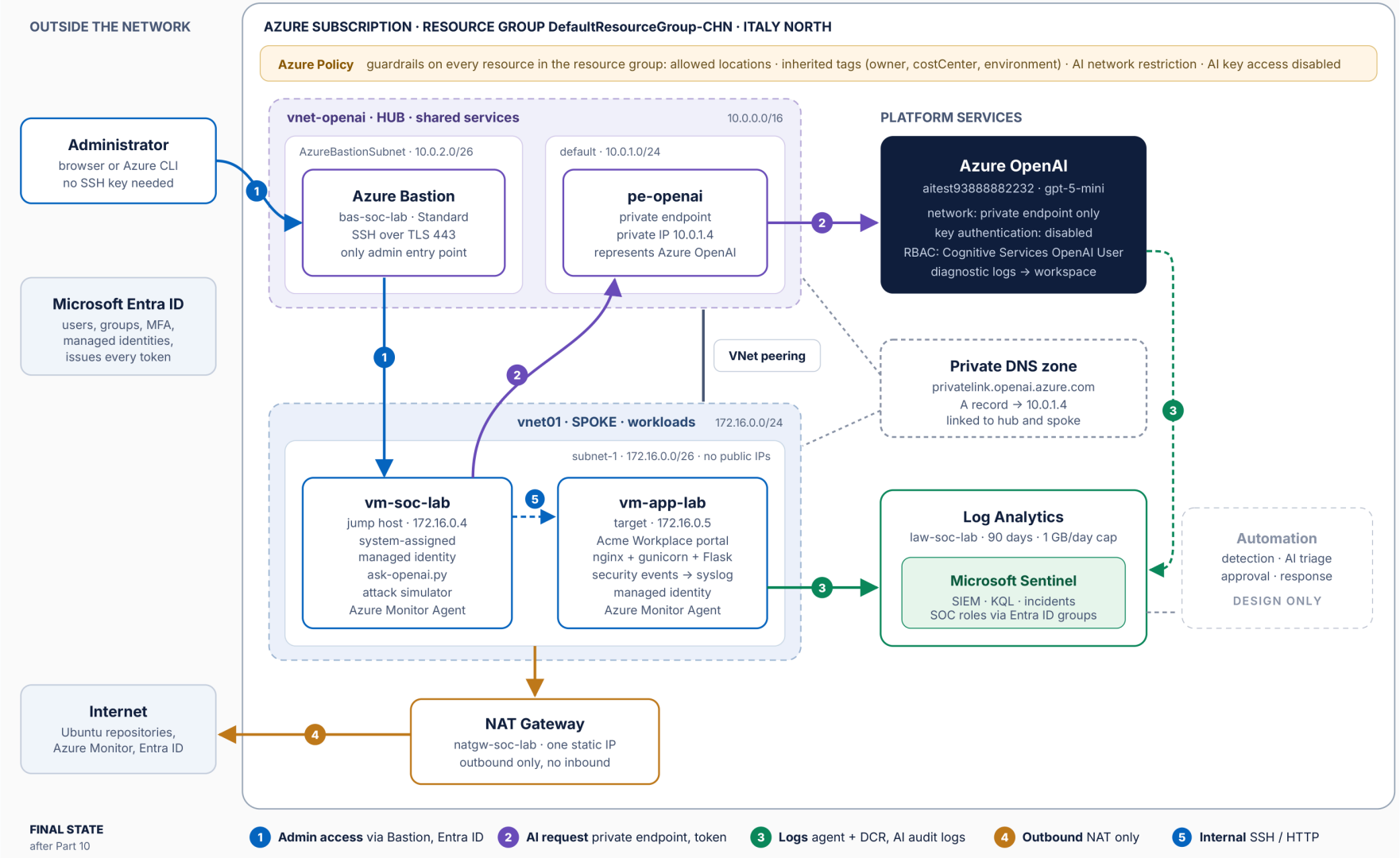
The network follows the hub-and-spoke model: shared services live in the hub, the workloads live in the spoke, and the two are joined by a peering and a private DNS zone. The workloads reach the internet only outbound, through a NAT Gateway.

No virtual machine has a public IP address. Administrators enter exclusively through Azure Bastion, signing in with their own Entra ID identity rather than with an SSH key.

A security log pipeline collects the events of the servers through the Azure Monitor Agent and a Data Collection Rule, while Azure OpenAI sends its own audit logs; everything lands in Log Analytics, where Microsoft Sentinel provides the SIEM layer.

Finally, the whole chain is validated end to end: a simulated attack against the application becomes visible in KQL queries within minutes, and the private model is able to assess the same events for the analyst.

Architecture



INTRODUCTION

Reading the architecture

The diagram shows the final state, after Part 10; during the build phase the jump host also has a temporary public IP and an SSH rule limited to the administrator's address. It is organized in three zones. On the left is everything outside the network: the administrator, Microsoft Entra ID, which issues every identity and token, and the internet destinations the servers need for updates and telemetry. In the center is the private network, made of the hub `vnet-openai` with Azure Bastion and the private endpoint, and the spoke `vnet01` with the two virtual machines. On the right are the platform services: Azure OpenAI, the private DNS zone and Log Analytics with Microsoft Sentinel. A band of Azure Policy across the top governs every resource in the resource group.

The administrator's path, in blue, starts with an Entra ID sign-in protected by MFA, continues over TLS on port 443 to Azure Bastion, and reaches the jump host on its private address through the peering, so no machine needs a public IP or an open SSH port.

The AI request, in purple, leaves the jump host towards the private endpoint: the private DNS zone resolves the Azure OpenAI hostname to the private address, and the call carries a token of the VM's managed identity, because API keys are disabled and public access is blocked.

The logs, in green, flow from both virtual machines to Log Analytics through the Azure Monitor Agent, authenticated with each VM's managed identity, while a diagnostic setting sends the Azure OpenAI audit logs to the same workspace.

The outbound traffic, in orange, leaves the spoke only through the NAT Gateway, from a single static IP, and nothing can connect back in. The internal flow, in dashed blue, is the jump host administering the application server over SSH and HTTP on the private network.

Design decisions

Governance first. Azure Policy turns the design rules into checks enforced by the platform rather than by memory: a resource in the wrong region is denied, a missing owner tag is added automatically, and an AI endpoint open to the internet or still accepting keys cannot be created.

Private model. Azure OpenAI is reachable only through a private endpoint, so the model is not exposed to the internet and a leaked credential alone is not enough to use it. Key-based authentication is disabled as well, so only Entra ID tokens are accepted.

Identities, not secrets. Workloads authenticate with managed identities: no secret is written to disk, and tokens are short-lived. Permissions follow least privilege, with an inference-only role for the model and roles assigned to Entra ID groups rather than to people.

Hub and spoke. The network uses the hub-and-spoke topology of the Microsoft Cloud Adoption Framework, with shared services in the hub and one spoke per workload. Outbound traffic goes through a NAT Gateway, so the workloads need no public IP and always leave from one known address.

Administration. Administrative access goes through Azure Bastion with Entra ID login on the virtual machines. No SSH port is exposed and no administrator depends on a personal SSH key; the build-phase key is temporary and removed in Part 10.

Separation and limits. The jump host and the target are separate machines, so administration and the monitored workload never share a role. The Data Collection Rule collects only security-relevant syslog facilities, and a daily cap, a retention period and a budget are defined before any data arrives.

INTRODUCTION

Threat model

Every control in this build exists to stop a specific threat, and each one ends with a test that proves it.

Threat	Control	Proven on
The model is called from the internet	Public access disabled, private endpoint, private DNS zone	Slides 27 and 45: 403 from outside, private address 10.0.1.4
A leaked API key is used	Key authentication disabled; only Entra ID tokens are accepted	Slide 26: `disableLocalAuth` returns true
A compromised client reconfigures the AI service	Managed identity with an inference-only role	Slides 49 and 51: role JSON, 200 on the data plane
An exposed SSH port is brute-forced	No public IPs; Azure Bastion with Entra ID login	Slide 94: `whoami` returns the user principal name
A compromised VM reaches the others	Explicit NSG rules on the target	Slide 69: only the jump host and Bastion are allowed
Attacks on the application go unseen	Structured events, Azure Monitor Agent, DCR, KQL in Sentinel	Slides 82, 84 and 85: pipeline check, attack in KQL
Attacker-written log text steers the model	The model returns text only; a human decides	Slide 54: design rule, not a test
A non-compliant resource is created	Azure Policy: Deny for the region and the AI settings	Slides 16 and 28: assignments and Deny
Costs run away	Daily ingestion cap, retention period, budget alerts	Slide 59: 1 GB per day cap

INTRODUCTION

Resources and naming

Everything lives in one resource group and one region, so the environment is governed, costed and removed as a single unit. Every name in this table can be changed (Customizable).

Resource	Name in this project	Notes
Resource group	`DefaultResourceGroup-CHN`	Region Italy North
Azure OpenAI / deployment	`aitest93888882232` / `gpt-5-mini`	Public access disabled; key auth disabled
Private endpoint / DNS zone	`pe-openai` / `privatelink.openai.azure.com`	Zone name Required
Hub / spoke VNets	`vnet-openai` / `vnet01`	Shared services / workloads
NAT Gateway / Azure Bastion	`natgw-soc-lab` / `bas-soc-lab`	Outbound / administrative access
Jump host / target VM	`vm-soc-lab` / `vm-app-lab`	Ubuntu 24.04; the jump host has a temporary public IP until Part 10, the target never has one
Log Analytics + Sentinel	`law-soc-lab`	Retention 90 days, daily cap
Data Collection Rule	`dcr-soc-lab-syslog`	Linux syslog, both VMs

Under the naming convention of slide 4 they would read `rg-soc-lab-itn-001`, `aoai-soc-lab-itn-001`, `law-soc-lab-itn-001` and so on.

Prerequisites

An Azure subscription in which you can create resources and role assignments, which in practice means the **Owner** role, or Contributor together with User Access Administrator, on the resource group. Part 6 also creates security groups in Microsoft Entra ID; in a personal tenant you can do this yourself, while in a company tenant it is normally done by the identity team.

Before starting, check in Azure AI Foundry that the model you want is available in the region you have chosen, because model availability differs from region to region.

On the tooling side, Azure Cloud Shell with Bash and a web browser are needed, together with a local terminal with an SSH client for the build-phase access of Parts 3 to 9, and the Azure CLI for the Bastion connections of Part 10.

Finally, create a **budget** in Cost Management with alerts at fifty, eighty and one hundred percent before deploying anything. It costs nothing and it is the simplest protection against an unexpected bill.

PART 0

Governance baseline

GOAL Create the resource group with ownership tags and assign Azure Policy guardrails before any resource exists.

PART 0

Rules versus policies · 0.1 Resource group

Most security settings in Azure are rules that configure a single resource: a network security group rule, the firewall of Azure OpenAI, a Data Collection Rule or a role assignment. Each of them is only as reliable as the person who configures that particular resource.

Azure Policy works at a different level. A policy is assigned to a whole scope, such as a resource group, a subscription or a management group, and from that moment it evaluates every existing and every new resource in the scope, whoever creates it and with whatever tool. Depending on its effect, it can report a resource that breaks the rule, block it before it is created, or correct it automatically.

```
RG=DefaultResourceGroup-CHN        # Customizable
LOC=italynorth                     # Customizable
az group create -n $RG -l $LOC \
  --tags environment=lab workload=soc \
         owner=<YOUR_NAME> costCenter=<COST_CENTER>
```

The first step creates the resource group that will hold every resource of the project. A single group is one scope for policies, one view for cost and one operation for teardown. Its tags record who owns the environment and which budget pays for it; an owner and an environment tag are the minimum most organizations require.

PART 0

How Azure Policy works

Azure Policy is built from three objects. A **definition** is a JSON rule with an *if* condition and a *then* effect; the built-in definition called Allowed locations, for example, says that if a resource's location is not in an approved list, the request must be denied. An **assignment** applies a definition to a scope and fills in its parameters. An **initiative** groups many definitions into one assignment, which is how organizations apply whole regulatory baselines at once.

Because assignments are inherited, a policy assigned to the resource group covers every resource created inside it later, whether through the portal, the Azure CLI or an infrastructure-as-code pipeline.

The **effect** decides what happens to a resource that breaks the rule. With **Deny**, Azure Resource Manager rejects the request before the resource exists. With **Audit**, the request succeeds but the resource is reported as non-compliant. With **AuditIfNotExists**, Policy checks whether a related condition is present. With **Modify**, Policy changes the request on the fly, for example by adding a missing tag.

Policies are evaluated at request time and again in a periodic compliance scan, whose results appear in Policy, Compliance. Effects that change resources need a managed identity on the assignment with a role that allows the change. Policy governs how resources are configured; it does not inspect traffic or model calls.

PART 0

0.2 Policy guardrails assigned to the resource group

WHY Turns the project's security decisions into rules the platform enforces automatically.

USED FOR Part 1 is checked against the AI policies; every resource gets the three tags.

Built-in policy	What it enforces	Effect
Allowed locations	Resources only in `italynorth`. Global resources such as private DNS zones are excluded	**Deny**: any other region is rejected at creation
Inherit a tag from the resource group if missing	Every resource receives the `owner`, `costCenter` and `environment` tags	**Modify**: the tag is added automatically
Azure AI Services resources should restrict network access	Azure OpenAI must not accept traffic from all networks	**Audit** while Part 1 is built, then **Deny** (step 1.7)
Azure AI Services resources should have key access disabled	API keys must be disabled on Azure OpenAI	**Audit** while Part 1 is built, then **Deny** (step 1.7)

The two AI policies start on Audit because the guide creates the Azure OpenAI resource first and secures it one control at a time, so that each control can be explained and tested on its own: with Deny, the creation in step 1.1 would be refused. Once Part 1 is complete, step 1.7 switches both to Deny, so a non-compliant AI resource cannot exist. In larger organizations the same assignments are made at management group level, so every subscription below inherits them.

PART 0

0.2 Assign the policies and validate

```
RG_ID=$(az group show -n $RG --query id -o tsv)
defid() { az policy definition list --query "[?displayName=='$1'].name | [0]" -o tsv; }
# Allowed locations (Deny)
az policy assignment create --name allowed-locations --scope "$RG_ID" \
  --policy "$(defid 'Allowed locations')" \
  --params '{"listOfAllowedLocations":{"value":["italynorth"]}}'
# Inherit the owner, costCenter and environment tags (Modify: needs its own managed identity)
for t in owner costCenter environment; do
  az policy assignment create --name "inherit-$t-tag" --scope "$RG_ID" \
    --policy "$(defid 'Inherit a tag from the resource group if missing')" \
    --params "{\"tagName\":{\"value\":\"$t\"}}" \
    --mi-system-assigned --location $LOC --identity-scope "$RG_ID" --role Contributor
  az policy remediation create -n "fix-$t-tag" --policy-assignment "inherit-$t-tag" -g $RG
done
# Azure AI Services: network access and key access (Audit now, Deny in step 1.7)
for p in "restrict network access" "key access disabled"; do
  DEF=$(az policy definition list \
    --query "[?contains(displayName,'Azure AI Services') && contains(displayName,'$p')].name | [0]" -o tsv)
  az policy assignment create --name "aisvc-$(echo $p | tr ' ' '-')" --scope "$RG_ID" --policy "$DEF"
done
```

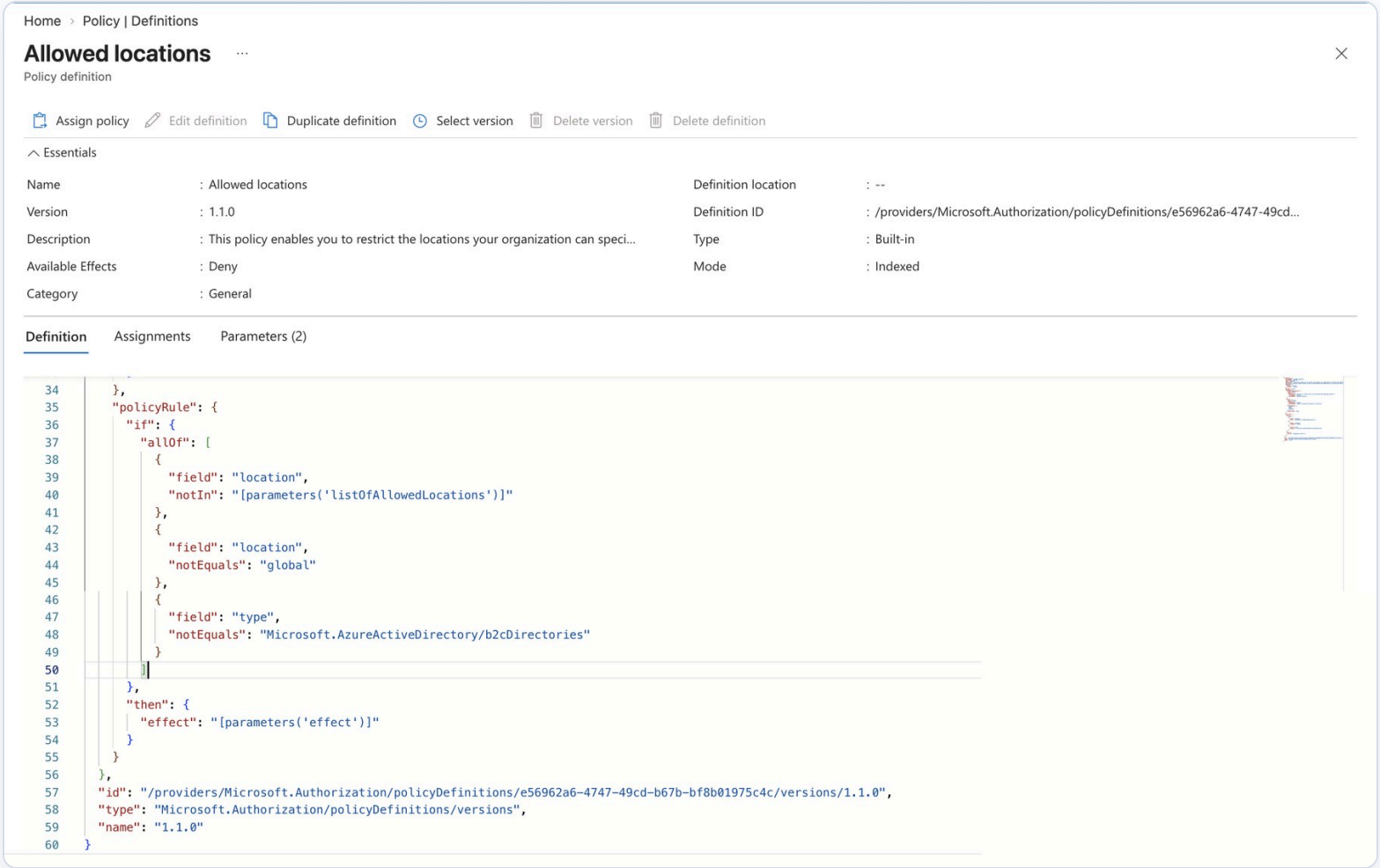
The tag-inheritance assignments are the only ones that receive a managed identity, because they are the only ones that change resources on your behalf; each remediation task adds the tag to resources that already exist. Open Policy, Compliance and filter on the resource group: the first evaluation completes within about thirty minutes.

PART 0

Inside a built-in definition: Allowed locations

Every built-in definition can be opened in Policy, Definitions, filtered on the Built-in type, to read its rule before it is assigned. The rule of Allowed locations is short: the *if* block matches any resource whose location is not in the parameter `listOfAllowedLocations`, that is not a global resource, and that is not an Azure AD B2C directory.

The *then* block reads the effect from a parameter, so it can be chosen at assignment time. For this policy the only available effect is **Deny**, which means a resource created in any other region is rejected before it exists.



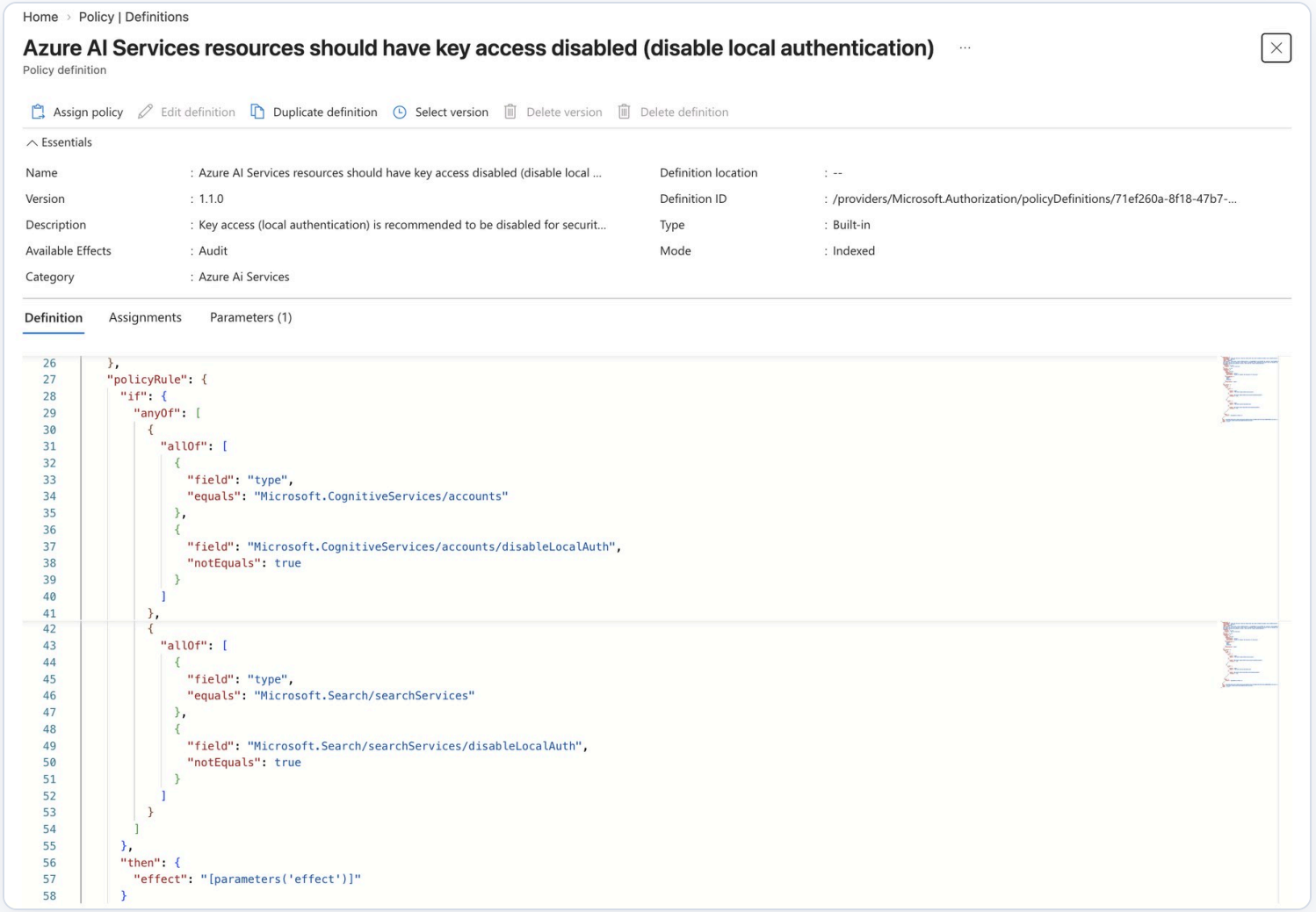
Allowed locations: rule and effect

PART 0

Inside a built-in definition: AI Services key access

This definition targets resources of type `Microsoft.CognitiveServices/accounts`, the type of every Azure OpenAI resource, and matches those whose `disableLocalAuth` property is not true, in other words those that still accept API keys. A second block applies the same check to Azure AI Search, so a single assignment covers both kinds of AI services.

Its default effect is Audit, which is why the resource appears as non-compliant until step 1.5 disables the keys. Step 1.7 then assigns it with the effect set to **Deny**.



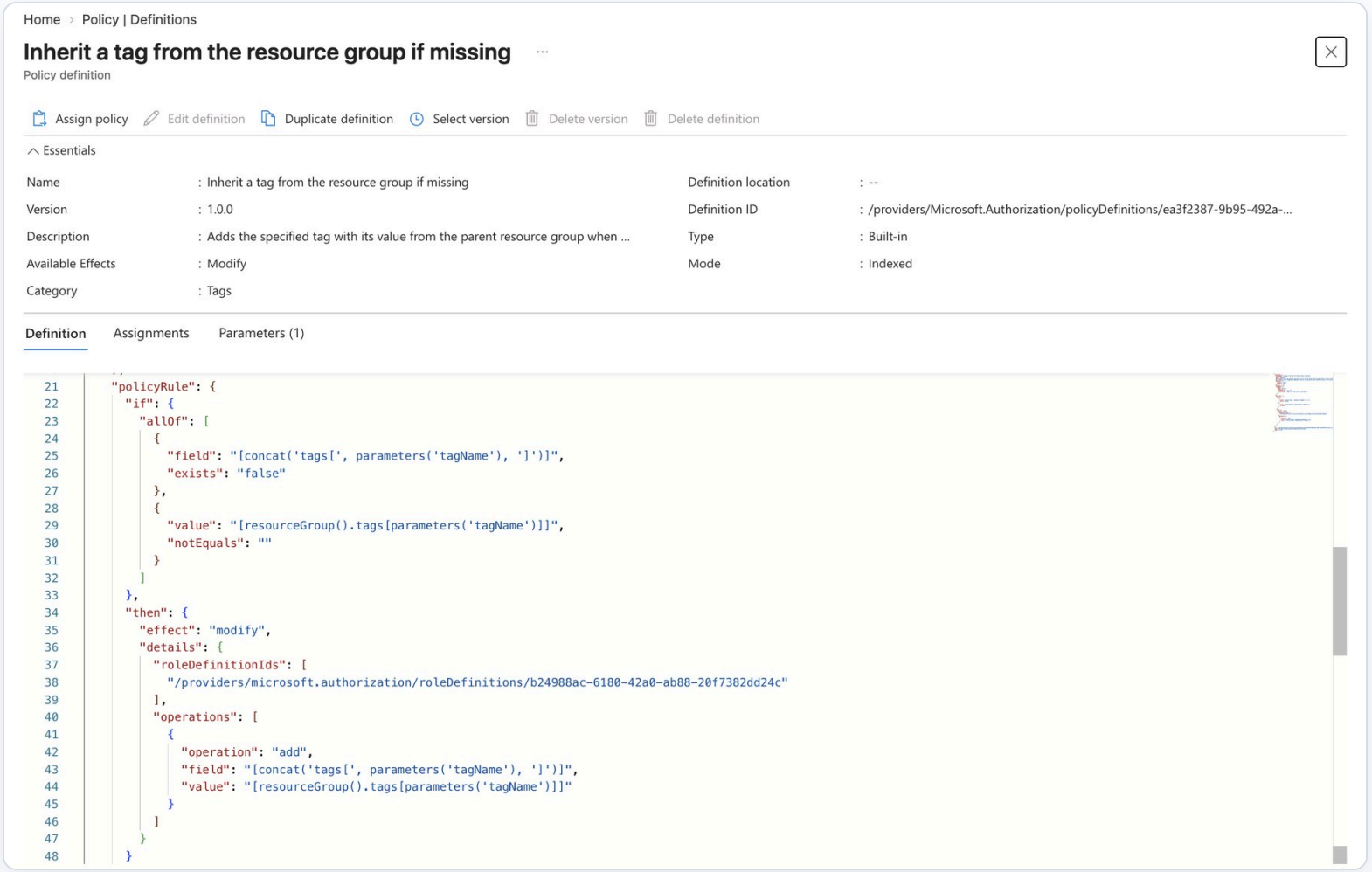
AI Services key access: matches accounts whose disableLocalAuth is not true

PART 0

Inside a built-in definition: tag inheritance

The rule matches a resource on which the tag named in `tagName` does not exist, while the resource group has a value for that tag. Instead of blocking the request, the *then* block uses the effect **modify**, with an operations list that adds the tag using the value of the resource group.

The `roleDefinitionIds` entry lists the role Policy needs to make that change on your behalf. The identifier `b24988ac-6180-42a0-ab88-20f7382dd24c` is the built-in Contributor role, which is why the assignment is created with a managed identity holding Contributor on the resource group.



Inherit a tag: Modify, with the Contributor role the managed identity needs

PART 1

Azure OpenAI behind a private endpoint

GOAL A deployed chat model that accepts traffic only from the private network and only with Entra ID authentication.

PART 1

How Azure OpenAI is protected: two planes

Every Azure OpenAI resource exposes two separate interfaces. The **control plane**, reached through `management.azure.com`, is where the resource is created, its network is configured, models are deployed and keys are listed; it is protected by Azure RBAC and governed by Azure Policy. The **data plane** is the endpoint `https://<name>.openai.azure.com`, where prompts are sent and completions come back, and it is this interface that the network settings, the private endpoint and the authentication settings protect.

Applications never call a model directly. They call a **deployment**, a named instance of a specific model version with its own throughput quota in tokens per minute; requests above that quota are throttled with HTTP 429, which also limits what a compromised client could consume.

Where the model processes requests depends on the deployment type. A Standard deployment runs inference in the resource's own region, a Global Standard deployment lets Microsoft route requests to any Azure region, and a Data Zone deployment keeps processing inside a data zone such as the European Union, the usual choice for EU data residency.

The data plane accepts two kinds of credentials. An **API key**, sent in the `api-key` header, identifies nobody, never expires and gives full access to whoever holds it. An **Entra ID token**, sent as a Bearer token for the audience `cognitiveservices.azure.com`, is tied to a specific identity, expires on its own, is authorized through RBAC and appears in the logs. Disabling local authentication leaves only the second kind.

PART 1

1.1 Create the Azure OpenAI resource

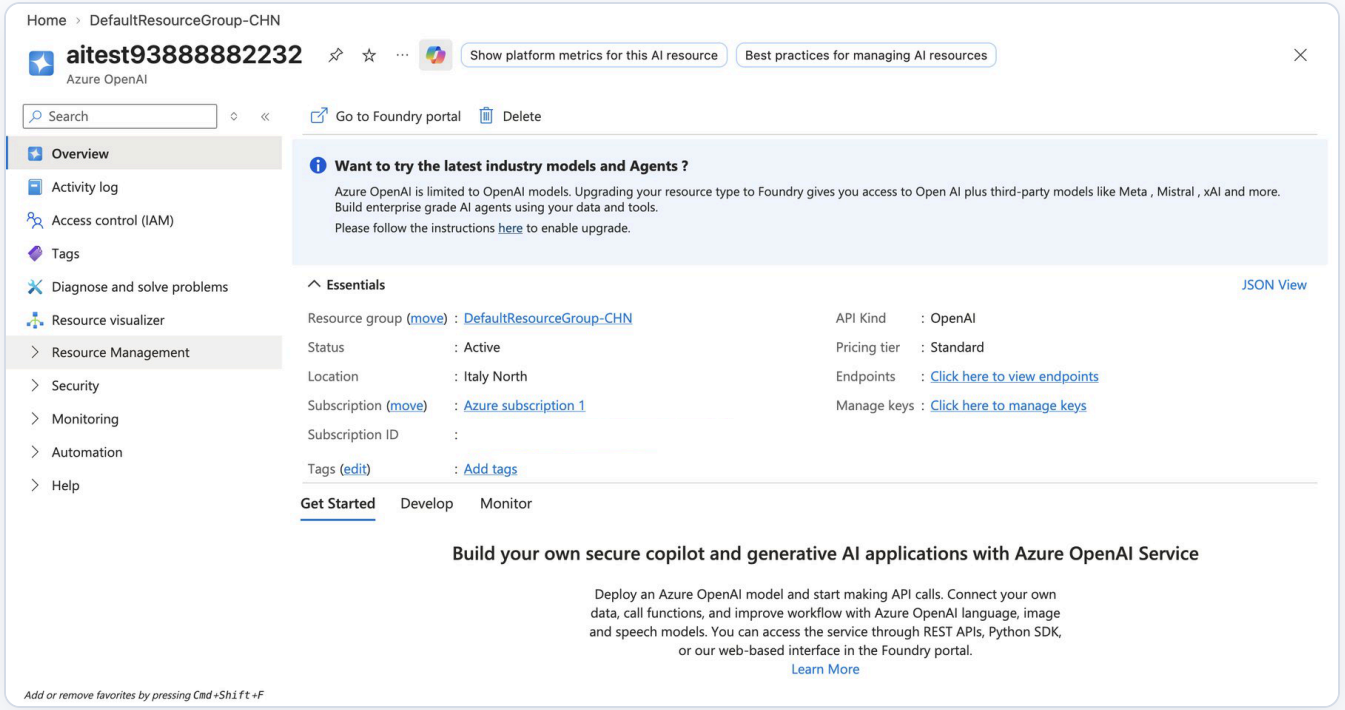
WHY The AI service the SOC uses to assess security events; everything else attaches to it.

USED FOR Model deployment, private endpoint, role assignments, diagnostic logs.

Azure OpenAI > Create

Field	Value	Type
Resource group	`DefaultResourceGroup-CHN`	Customizable
Region	Italy North. Must offer the model you need	Customizable
Name	Globally unique: it becomes the endpoint hostname	Customizable
Pricing tier	Standard S0	Required
Network	Closed in step 1.3	Required

The resource name becomes the custom subdomain that Entra ID authentication requires. Standard S0 is pay-per-token, with no fixed monthly cost.



Resource overview, captured before the tag policy had added its tags

PART 1

1.2 Deploy the model

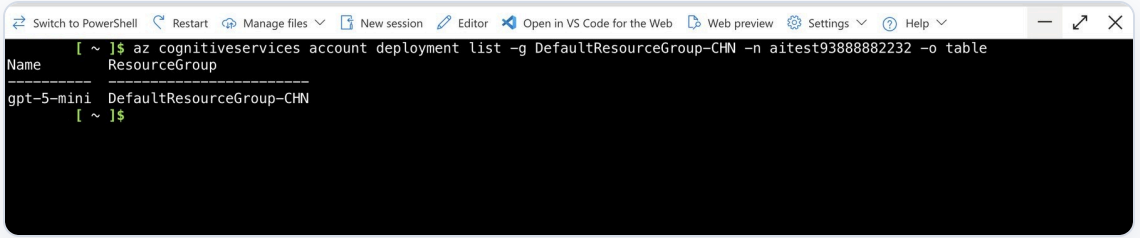
WHY An account alone cannot answer: a deployment exposes one model version under a name, with its own quota.

USED FOR The scripts and the playbook call the model by this name.

Azure AI Foundry > Deployments > Deploy model

Field	Value	Type
Model	gpt-5-mini, version 2025-08-07. Any chat model works with the scripts	Customizable
Deployment name	`gpt-5-mini`, referenced by `AOAI_DEPLOYMENT`	Customizable
Deployment type	Standard, Global Standard or Data Zone (keeps processing in the EU data zone)	Customizable
Tokens per minute	Throughput quota. A low value is enough for a lab	Customizable

```
az cognitiveservices account deployment list \
  -g DefaultResourceGroup-CHN -n aitest93888882232 -o table
```



Verified from Cloud Shell

PART 1

1.3 Close the public endpoint · 1.4 Create the private endpoint

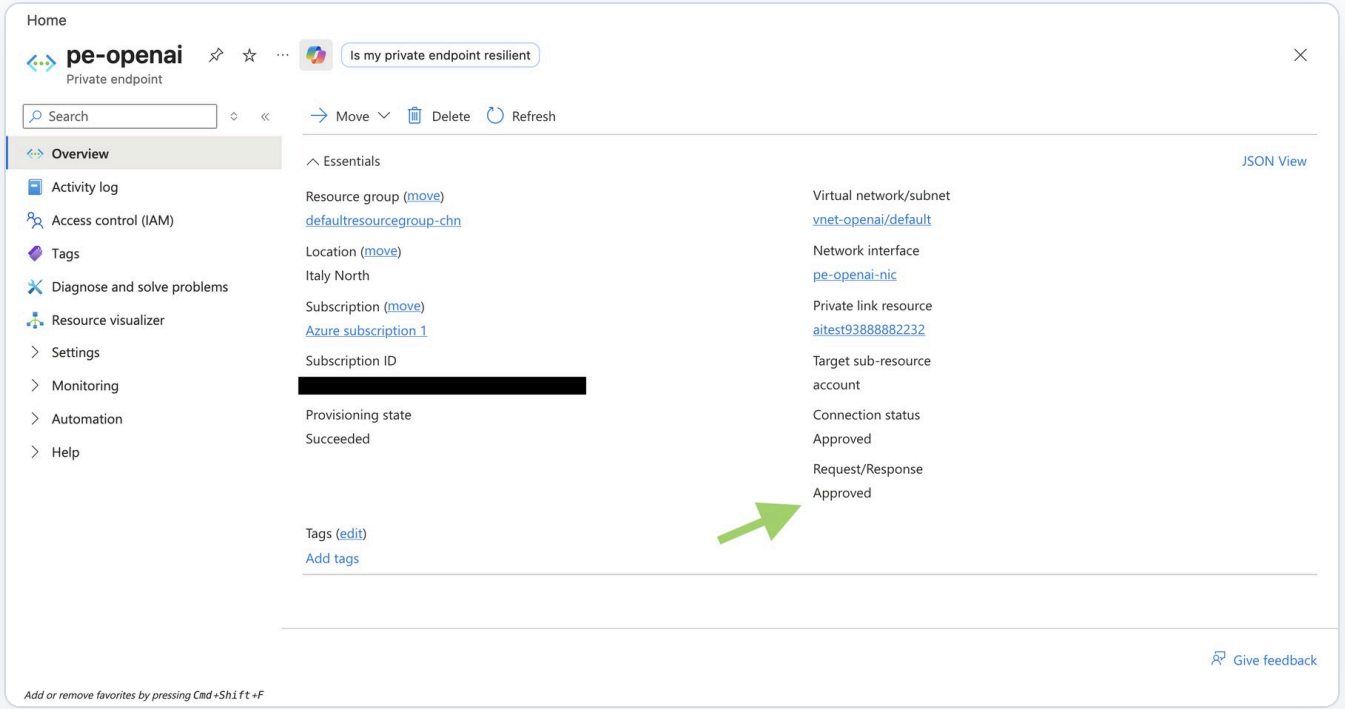
WHY Prompts and security data must not travel over the internet, and the model must be unreachable even with a valid credential.

USED FOR From now on the private endpoint is the only path to the model.

In the Networking page of the resource, on the **Firewalls and virtual networks** tab, set the access to **Disabled** and save. The public endpoint now refuses every request, so the private endpoint created next becomes the only path to the model.

On the **Private endpoint connections** tab, create the private endpoint with these values. When the deployment finishes, the connection state must read **Approved**.

Setting	Value	Type
Name	`pe-openai`	Customizable
Virtual network / subnet	`vnet-openai` / `default` (the hub)	Customizable
Target sub-resource	`account`	Required
Private DNS integration	Yes, zone `privatelink.openai.azure.com`	Required



pe-openai, Approved

PART 1

How the private endpoint and private DNS work

A private endpoint is the Azure Private Link mechanism for bringing a service into your network. It is a network interface that receives a private address from the subnet, 10.0.1.4 in this project, and every packet sent to that address is delivered to the Azure OpenAI resource over the Microsoft backbone, never across the internet. The connection works in one direction only: clients in the network can reach the service, but the service can never open a connection into the network.

Disabling public network access closes the public endpoint completely: every request that does not arrive through an approved private endpoint is refused, whatever its source, and no IP range, virtual network rule or trusted-service exception can reopen it.

For any of this to work, clients must resolve the resource name to the private address, and this is the job of the private DNS zone. Public DNS answers the name `aitest93888882232.openai.azure.com` with an alias to `aitest93888882232.privatelink.openai.azure.com`. Inside a virtual network linked to the private zone, that second name resolves to the private endpoint; anywhere else it resolves to the public address, where the request is refused with a 403.

This behavior is called split-horizon DNS: the same hostname gives a different answer inside and outside the network, so applications keep using the normal endpoint URL without any change. A virtual network uses the zone only if it is linked to it, which is why the spoke receives its own link in Part 2.

PART 1

1.5 Disable key-based authentication

WHY Keys leak into scripts, tickets and chats. With keys off, every call is tied to an Entra ID identity.

USED FOR Every caller in the project authenticates with Entra ID.

Azure OpenAI creates two API keys by default. Even when nobody uses them they are a standing risk, because anyone who obtains a key can call the model. The portal has no switch for this setting, so it is set from Cloud Shell:

```
OAI_ID=$(az cognitiveservices account show \
  -g DefaultResourceGroup-CHN -n aitest93888882232 --query id -o tsv)
az resource update --ids "$OAI_ID" --set properties.disableLocalAuth=true
# Verify: must return true
az cognitiveservices account show \
  -g DefaultResourceGroup-CHN -n aitest93888882232 --query properties.disableLocalAuth
```

The keys still exist on the control plane, but any request that presents one is rejected, even from inside the private network. From now on every call is tied to an Entra ID identity, appears in the logs under that identity, and can be revoked by removing a single role assignment. The Part 0 policy that requires key access to be disabled now reports the resource as compliant.

PART 1

1.6 Validate that public access is blocked

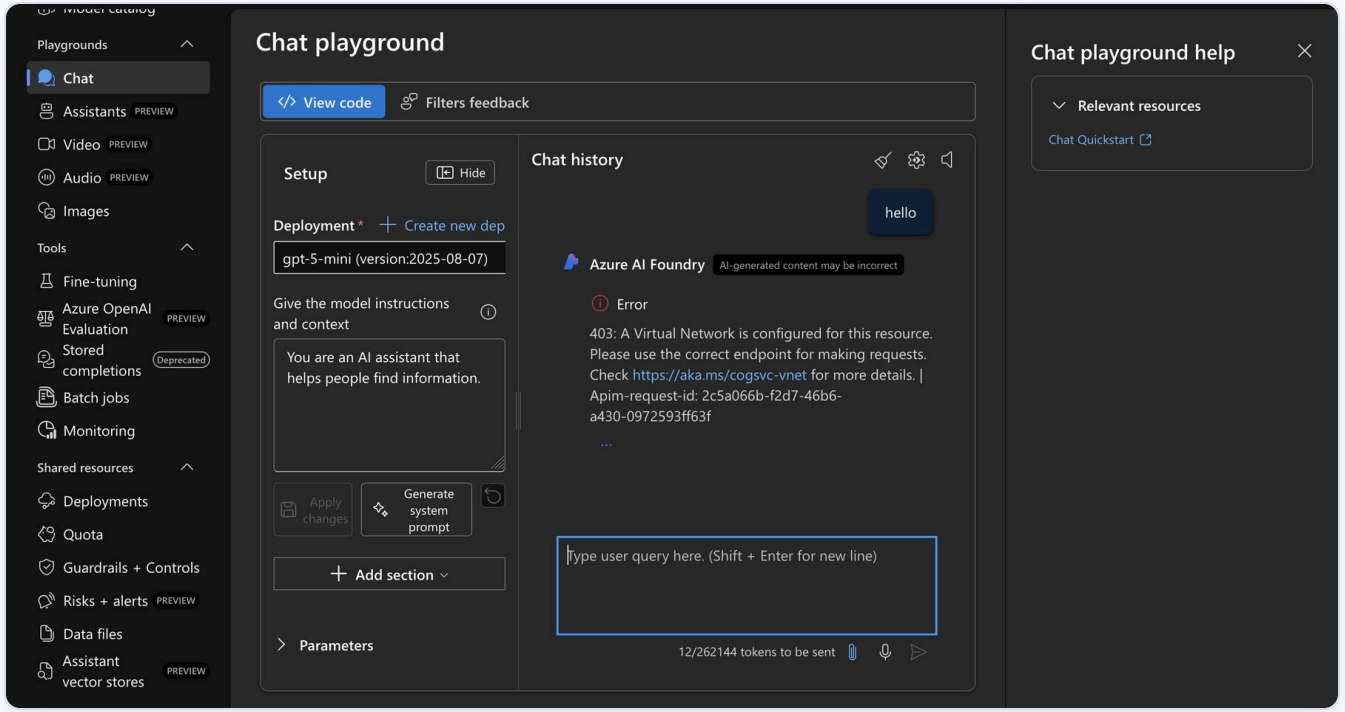
WHY A control that is not tested is only an assumption.

USED FOR Evidence that the model is usable only through the private path.

The last step of Part 1 tries to use the model from outside the network. Open the deployment in the Azure AI Foundry chat playground on your own workstation and send a message.

The playground runs in your browser and calls the data plane directly from your public IP address. Because public network access is disabled, the service answers **403 – A Virtual Network is configured for this resource**.

Opening the resource in the portal still works, because that uses the control plane, which is authorized by your role and not by the network settings.



The playground on the administrator's computer is refused with 403

PART 1

1.7 Switch the AI policies to Deny

WHY Audit reports a non-compliant AI resource; Deny stops it from existing.

USED FOR Every AI resource in the resource group, from now on.

The two AI policies were assigned with Audit in Part 0 so that each control of Part 1 could be built and tested on its own. Once public access is disabled and key authentication is off, Audit has done its job, but it only reports a non-compliant resource after the resource exists. With **Deny**, Azure Resource Manager rejects any request that would create an Azure AI Services resource open to all networks or with key access enabled, or that would change an existing one back to that state.

```
for p in "restrict network access" "key access disabled"; do
  NAME="aisvc-$(echo $p | tr ' ' '-')"
  DEF=$(az policy definition list \
    --query "[?contains(displayName,'Azure AI Services') && contains(displayName,'$p')].name | [0]" -o tsv)
  az policy assignment delete --name "$NAME" --scope "$RG_ID"
  az policy assignment create --name "$NAME" --scope "$RG_ID" --policy "$DEF" \
    --params '{"effect":{"value":"Deny"}}'
done
```

In the portal the same change is made in Policy, Assignments, **Edit assignment**, Parameters, Effect = Deny. To validate, try to create an Azure OpenAI resource with public network access set to all networks in the resource group: the request must fail with `RequestDisallowedByPolicy`.

PART 2

Hub-and-spoke network, private DNS and outbound connectivity

GOAL Workloads reach the private endpoint, resolve its private IP, and reach the internet only outbound through a NAT Gateway.

PART 2

2.1 Address plan

WHY Peered networks cannot overlap, and Bastion needs a dedicated subnet: planning avoids rebuilding.

USED FOR Peering, private endpoint, VMs and Bastion.

Network / subnet	Role	Range	Type
`vnet-openai` (hub)	Shared services	`10.0.0.0/16`	Customizable
`vnet-openai/default`	Private endpoint	`10.0.1.0/24`	Customizable
`vnet-openai/AzureBastionSubnet`	Azure Bastion (Part 10)	`10.0.2.0/26`	Name and min. /26 Required
`vnet01` (spoke)	Workloads	`172.16.0.0/24`	Customizable
`vnet01/subnet-1`	Both VMs	`172.16.0.0/26`	Customizable

The ranges are written in CIDR notation: a /16 holds 65,536 addresses, a /24 holds 256 and a /26 holds 64, of which Azure reserves five in every subnet. The one hard rule is that peered networks must not overlap; in an organization, the ranges come from the central IP address management plan. Azure Bastion needs a subnet named exactly `AzureBastionSubnet`, of size /26 or larger. Hub-and-spoke keeps shared services and DNS central and lets one Azure OpenAI resource serve many spokes without touching the others.

PART 2

How peering and NSGs control traffic

Virtual networks in Azure are isolated by default. A VNet peering connects two of them so that resources communicate through private addresses, with the same latency and bandwidth as inside a single network and without leaving the Microsoft backbone. A peering is made of two links, one in each direction; in this design both allow access to the remote network, while gateway transit and forwarded traffic stay off, because there is no VPN gateway or appliance in the hub.

Peering is deliberately not transitive. If a second spoke were peered to the hub, it could not reach `vnet01` through the hub without additional routing, which keeps every workload isolated from the others by default.

Traffic that crosses a peering is still filtered by network security groups. An NSG is a stateful packet filter attached to a subnet or a network interface; its rules are evaluated in order of priority, from 100 to 4096, the first match decides, and the return traffic of an allowed connection is permitted automatically.

Every NSG ends with three default rules. `AllowVnetInBound` (65000) accepts traffic from the virtual network and peered networks, `AllowAzureLoadBalancerInBound` (65001) accepts health probes, and `DenyAllInBound` (65500) refuses everything else, including the whole internet.

The first default would let every machine in the hub and the spoke reach every port of the others. The target's NSG therefore adds explicit rules (slide 69): SSH and HTTP only from the jump host, SSH from the Bastion subnet, and a deny for the rest of the virtual network.

PART 2

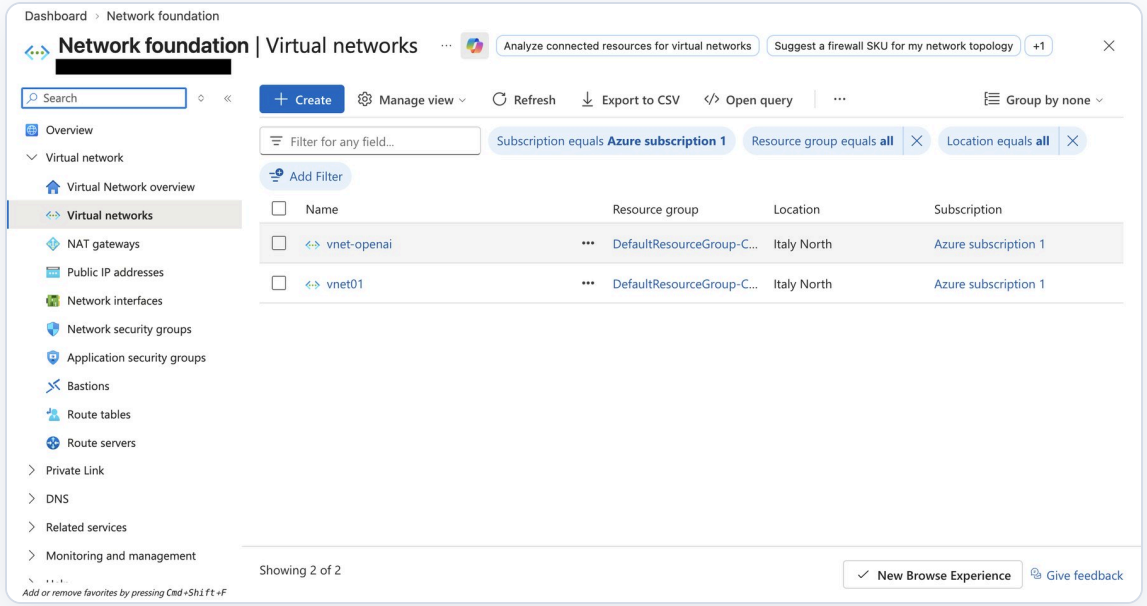
2.2 Peer the two networks

WHY VNets are isolated by default; the peering joins spoke and hub over the Microsoft backbone.

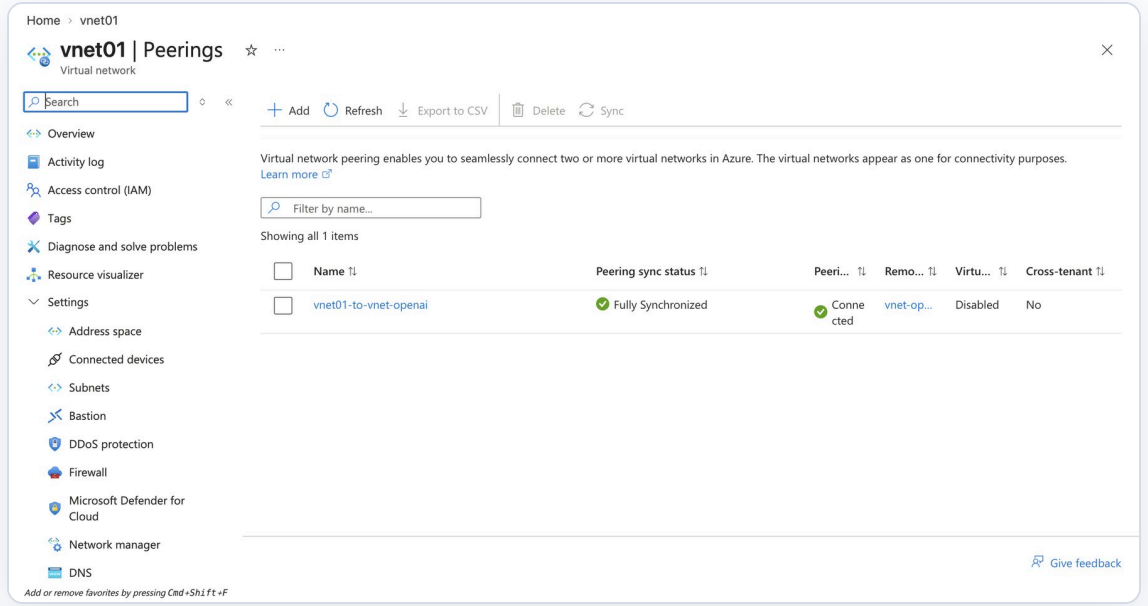
USED FOR Jump host → private endpoint (Part 4); Bastion → VMs (Part 10).

Open `vnet01`, then Settings, Peerings and Add. On the remote side, name the link `vnet-openai-to-vnet01`, select `vnet-openai` and allow access; on the local side, name the link `vnet01-to-vnet-openai` and allow access as well. Leave gateway transit disabled, because the design has no VPN gateway.

Wait until the peering status reads **Connected** and the synchronization status **Fully Synchronized**. From then on the jump host can reach the private endpoint in the hub, and Azure Bastion in the hub can reach the virtual machines.



The two virtual networks



Peering connected

PART 2

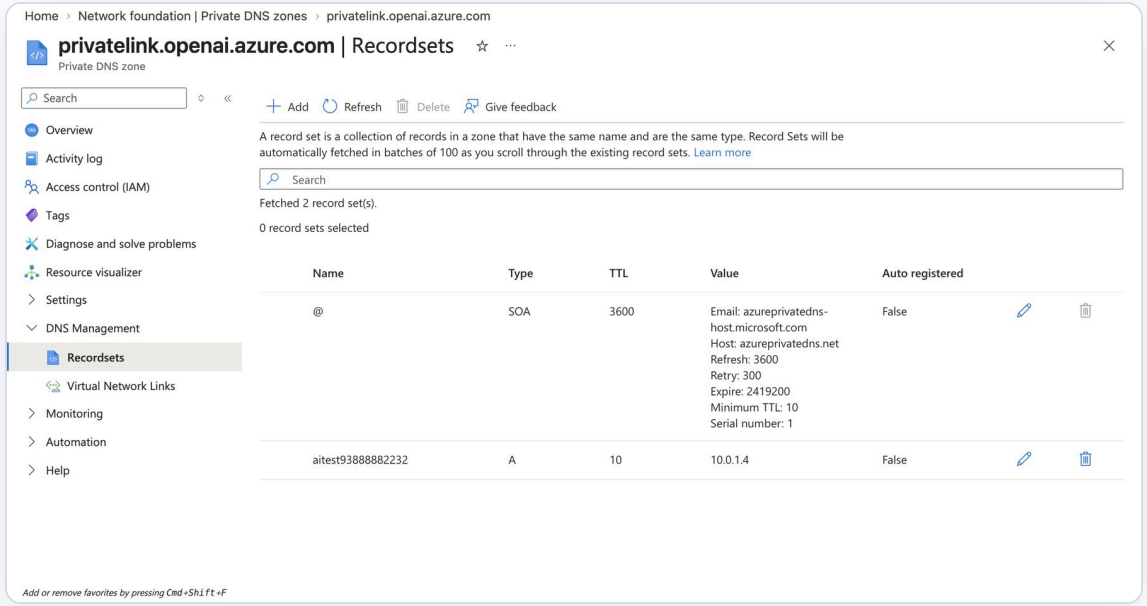
2.3 Link the private DNS zone to the spoke

WHY A VNet resolves a private zone only if linked: without it, the VMs would resolve the public IP and get 403.

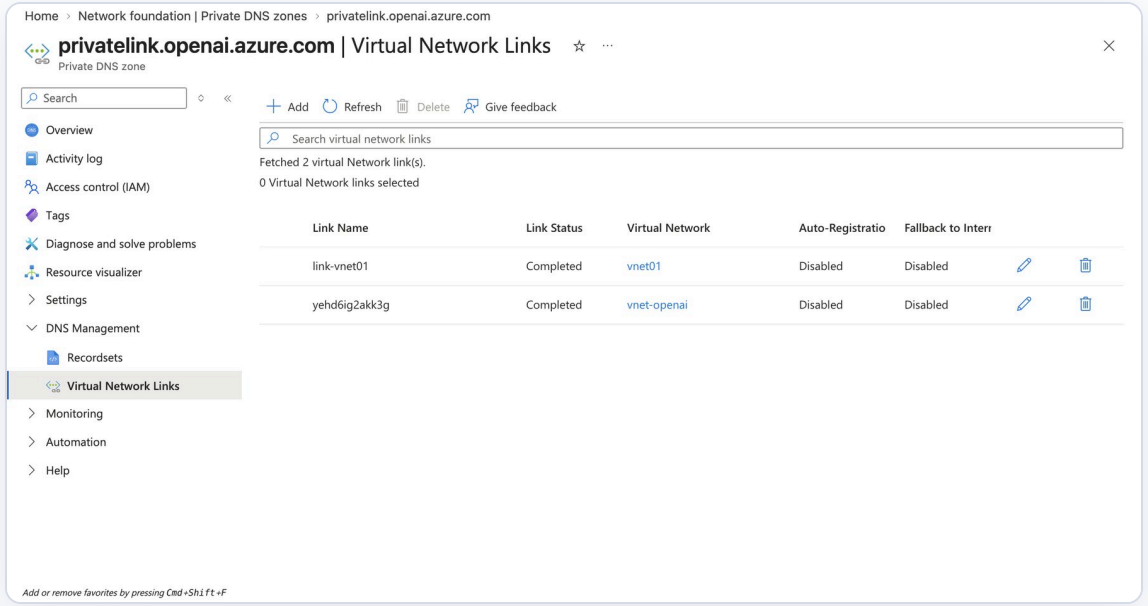
USED FOR Every call from the spoke resolves to the private endpoint.

The private endpoint wizard created the zone `privatelink.openai.azure.com` with an A record that maps the resource name to 10.0.1.4, but linked it only to the hub. Every VM uses the Azure-provided resolver at 168.63.129.16, which answers from a private zone only for the networks linked to it.

In the zone, open **Virtual Network Links** and add a link named `link-vnet01` for `vnet01`, with auto registration turned off: auto registration creates records for VM hostnames, which have no place in a private endpoint zone. Both links must end in the **Completed** state.



A record → 10.0.1.4



Zone linked to both VNets

PART 2

How the NAT Gateway works

A NAT Gateway performs source network address translation. When a VM in the associated subnet opens a connection to the internet, the gateway replaces the VM's private source address with its own public address and a source port, and translates the replies back. Because these translations exist only for connections that started inside the network, the gateway never forwards unsolicited traffic from the internet.

Without an explicit outbound method, Azure VMs historically used default outbound access, a shared address that the customer does not own or control. Microsoft is retiring that behavior for new virtual networks and recommends the NAT Gateway as the explicit, controlled alternative.

Once a subnet has a NAT Gateway, all of its outbound traffic uses it, with priority over public IPs attached to individual VMs. Each public address provides 64,512 source ports, and every machine leaves through the same static address, which partners and SaaS providers can add to their allowlists.

A NAT Gateway is a zonal resource, so it lives in the same availability zone as the VMs it serves. It translates traffic but does not inspect it: organizations that need to filter outbound traffic by domain add a firewall in the hub.

PART 2

2.4 Add a NAT Gateway for outbound connectivity

WHY VMs need outbound access but must not be reachable: NAT allows only outbound, from one static IP.

USED FOR Package installs, agent traffic, and after Part 10 the jump host's only way out.

The virtual machines need outbound access to install packages and to send their logs, but they must not be reachable from the internet. Instead of giving each VM its own public IP, the workload subnet receives a NAT Gateway, so every outbound connection leaves from one static address and no inbound connection can be initiated.

Setting	Value	Type
Availability zone	1, the same as the VMs	Customizable
Idle timeout	4 minutes	Customizable
Public IP	One static Standard IP, `pip-natgw-soc-lab`	Customizable

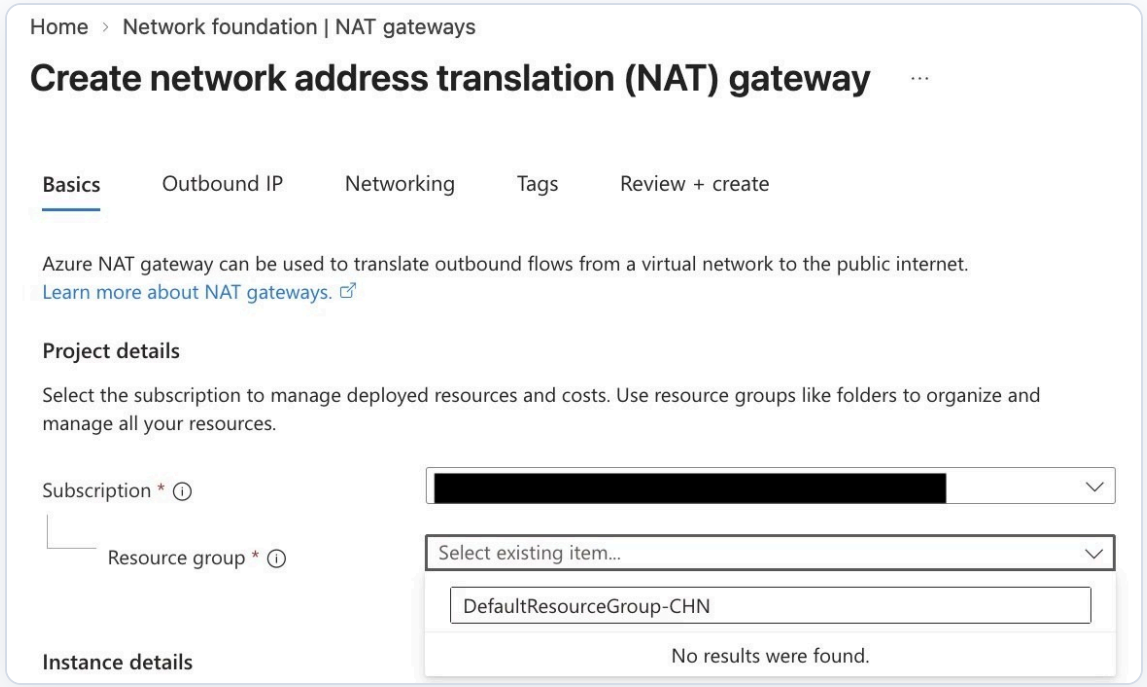
Once `vm-app-lab` exists in step 7.2, `curl -s ifconfig.me` on it must print the public IP of `pip-natgw-soc-lab`.

```
RG=DefaultResourceGroup-CHN
az network public-ip create -g $RG -n pip-natgw-soc-lab \
  --sku Standard --allocation-method Static --zone 1
az network nat gateway create -g $RG -n natgw-soc-lab \
  --public-ip-addresses pip-natgw-soc-lab \
  --idle-timeout 4 --zone 1
az network vnet subnet update -g $RG --vnet-name vnet01 \
  -n subnet-1 --nat-gateway natgw-soc-lab
```

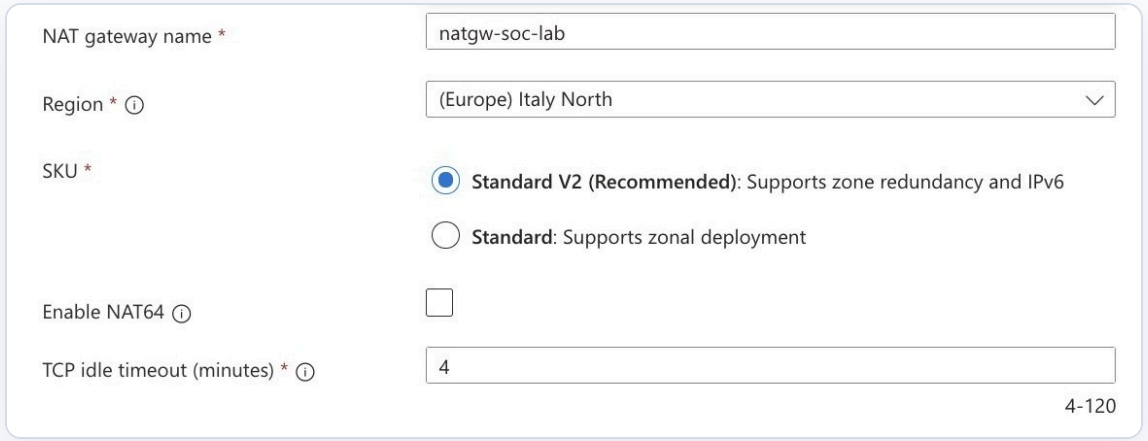
PART 2

2.4 The NAT Gateway in the portal · Basics

NAT gateways > Create > **Basics**: name `natgw-soc-lab`, region, SKU (Standard V2 or Standard, either works) and a 4-minute idle timeout.



Basics: project details

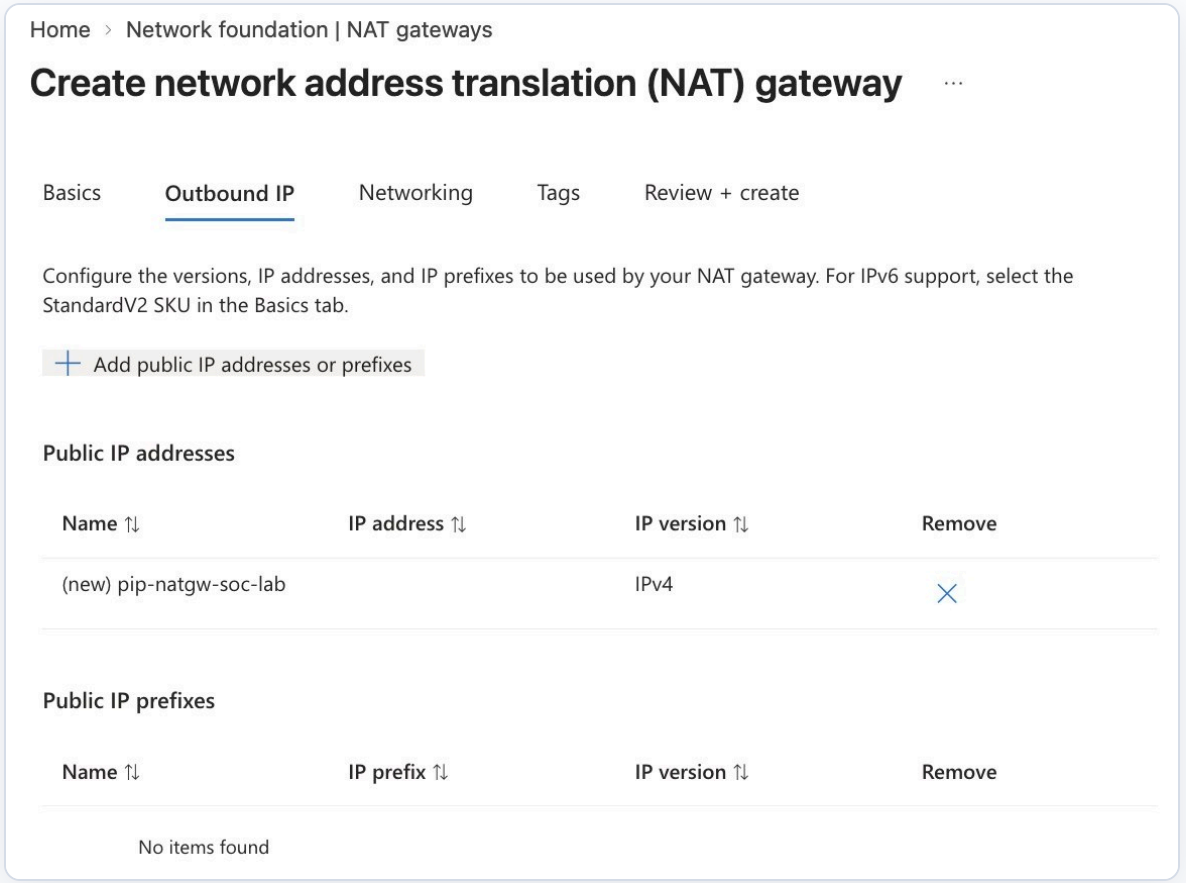


Basics: instance details

PART 2

2.4 The NAT Gateway in the portal · Outbound IP

The **Outbound IP** tab adds a new public address, `pip-natgw-soc-lab`, which every outbound connection will use. The association with `subnet-1` is set on the **Networking** tab.



Outbound IP

PART 3

Jump host with a managed identity

GOAL The administrative VM inside the spoke and the client that calls Azure OpenAI with its own identity. It starts with temporary SSH key access, replaced by Bastion and Entra ID login in Part 10.

PART 3

How the jump host is protected

When the **system-assigned managed identity** is enabled, Azure creates a service principal in Microsoft Entra ID that represents this specific VM and shares its lifecycle, so it disappears when the VM is deleted. The VM can request tokens for that identity from the local metadata service, and the identity can receive Azure roles exactly like a user. A user-assigned identity is the alternative when several resources need to share one identity that outlives them.

Trusted launch adds Secure Boot, which allows only signed bootloaders and kernels to start, and a virtual TPM that measures the boot chain, protecting the machine against bootkits and rootkits.

The size name describes the machine: in D2alds_v7, D is the general-purpose family, 2 the number of vCPUs, a stands for AMD, l for low memory per vCPU, d for a local temporary disk and s for Premium SSD support.

Access to the jump host changes during the project. In the build phase, Parts 3 to 9, Bastion and Entra ID login do not exist yet, so the first sessions use an SSH key generated by Azure, through a public IP restricted to the administrator's address. Azure writes the public key into the VM, while the private key stays on the administrator's computer; password login is disabled. This is temporary, bootstrap access.

The final access model is built in Part 10. Administrators connect through Azure Bastion and authenticate with their Microsoft Entra ID account, protected by MFA, through a short-lived certificate instead of a key file. The connection still uses SSH, but over the private network; the public IP, the SSH rule and the build-phase key are then removed.

PART 3

3.1 Create vm-soc-lab · Basics and Disks

Virtual machines > Create > Azure virtual machine. Settings not listed keep their defaults.

WHY One controlled administrative entry point; the managed identity lets it call Azure with no stored secret.

USED FOR AI calls (Parts 4–5), target administration (Part 7), attack simulation (Part 9).

Field	Value	Type
Region / availability zone	Italy North, zone 1 (one zone only, or the portal creates one VM per zone)	Customizable
Security type	Trusted launch, Secure Boot and vTPM enabled	Required
Image	Ubuntu Server 24.04 LTS – x64 Gen2 (commands assume Ubuntu)	Customizable
Size	Standard D2alds_v7 (2 vCPU, 4 GiB). Choose the smallest size available in your region and quota	Customizable
Authentication	SSH public key, user `azureuser`, new RSA key pair `vm-soc-lab_key`	Required
Public inbound ports	SSH (22) for the build phase: restricted in 3.2, removed in Part 10	Required
OS disk type	Premium SSD (LRS). Standard SSD is enough for a lab	Customizable
Delete disk with VM / encryption	Enabled / platform-managed keys (customer-managed in regulated environments)	Customizable

PART 3

3.1 Create vm-soc-lab · Networking, Management, Monitoring

Field	Value	Type
Virtual network / subnet	`vnet01` / `subnet-1`. Do not keep the network proposed by the wizard	Required
Public IP	New, `vm-soc-lab-ip`, for the build phase. Removed in Part 10	Required
NIC network security group	Basic, SSH (22)	Required
Delete public IP and NIC with VM	Enabled (off by default): prevents orphaned, billed resources	Required
System assigned managed identity	Enabled: the VM authenticates to Azure OpenAI without any secret	Required
Login with Microsoft Entra ID	Enabled in Part 10	Required
Auto-shutdown / patch orchestration	Not offered in every region / Azure-orchestrated patching	Customizable
Boot diagnostics / OS guest diagnostics	Managed storage / Off: logs come from the DCR in Part 8	Required

After **Review + create** and **Create**, the portal offers the private key (.pem) for download exactly once, so save it before closing the page.

PART 3

3.2 Restrict SSH · 3.3 Connect

WHY An SSH port open to the internet is scanned within minutes; one allowed IP reduces exposure to almost zero.

USED FOR The build phase only: removed in Part 10.

The wizard created an inbound rule called SSH that accepts port 22 from any address. In `vm-soc-lab`, **Networking, Network settings**, change the source of that rule to **My IP address**. The rule then matches only your current public IP, and every other connection falls through to the default deny rule.

> **Temporary access.** The public IP and this rule exist only for Parts 3 to 9. Part 10 removes both, together with the SSH keys, and from then on the only way in is Azure Bastion with Entra ID login.

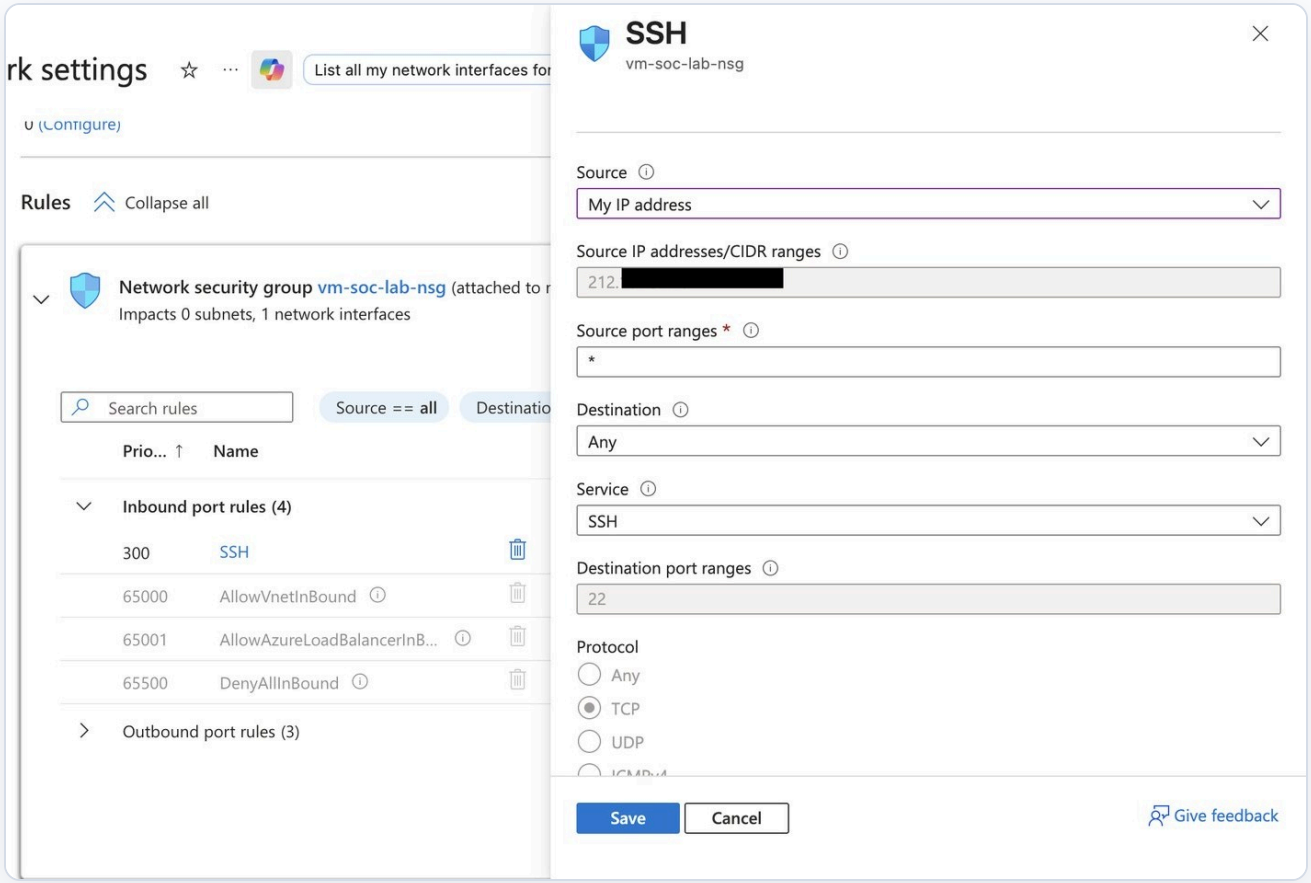
```
# on your computer
chmod 400 ~/Downloads/vm-soc-lab_key.pem
ssh -i ~/Downloads/vm-soc-lab_key.pem \
  azureuser@<JUMP_PUBLIC_IP>
```

`chmod 400` makes the key readable only by your user, because SSH refuses keys with broader permissions. The second command opens the first session on the jump host.

PART 3

3.2 The SSH rule restricted to one source IP

`vm-soc-lab` > Networking > Network settings: the inbound rule **SSH** (priority 300) with **My IP address** as source. Every other address falls through to the default deny rule.

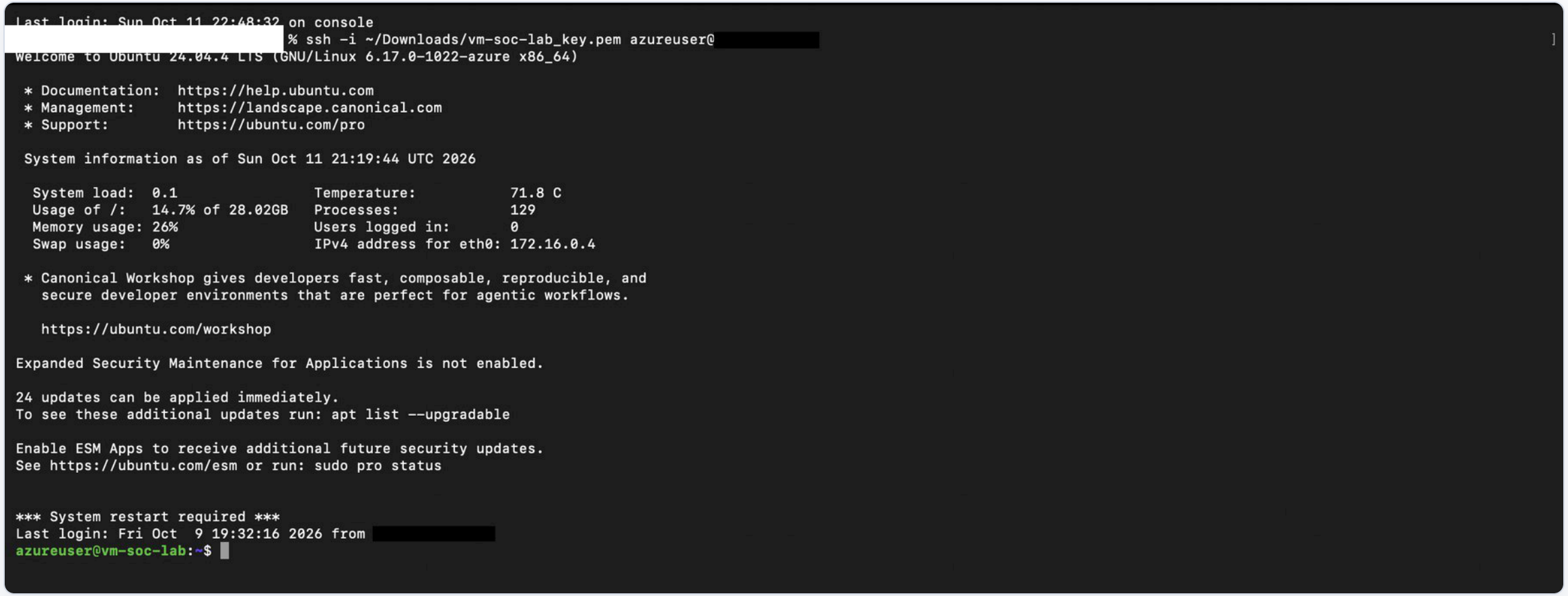


SSH restricted to one source IP

PART 3

3.3 The first session on vm-soc-lab

The first build-phase session on the jump host, opened from your computer with the downloaded key.



Session on vm-soc-lab

PART 3

3.4 Validate private name resolution

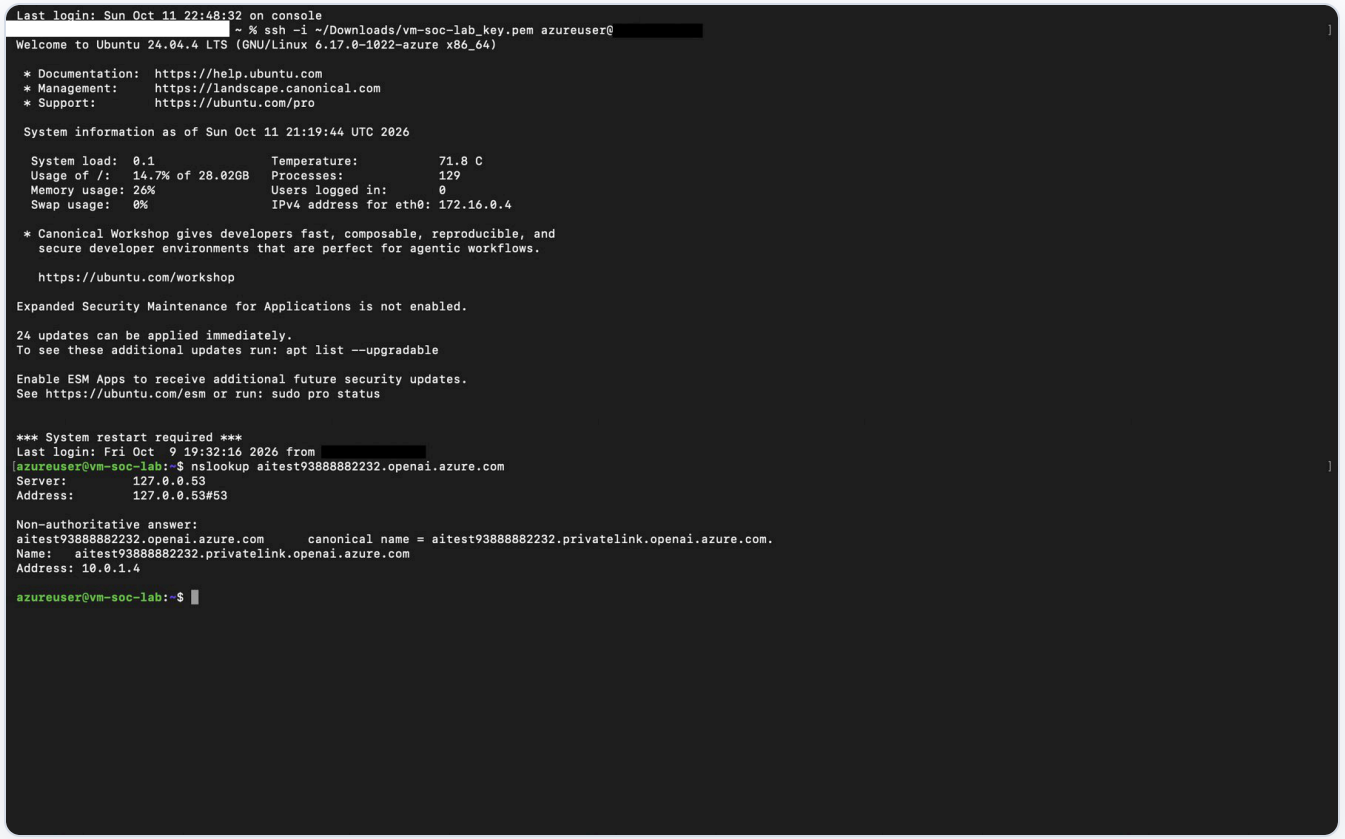
WHY Proves peering and DNS before authentication is involved: a public answer means every call fails.

USED FOR The token and API tests of Part 4.

```
# on vm-soc-lab
sudo apt update && sudo apt install -y dnsutils
nslookup aitest93888882232.openai.azure.com
```

Before any authentication is involved, the jump host checks that it can find the private endpoint. The lookup must follow the alias to the privatelink name and return the private address **10.0.1.4**, which proves at once that the peering and the DNS zone link work.

The server shown as 127.0.0.53 is the local resolver of Ubuntu, which forwards the query to the Azure resolver. If a public address comes back instead, the zone link of step 2.3 is missing, and every later call would fail with a 403.



The jump host resolves the endpoint to 10.0.1.4 (build-phase session)

PART 4

Keyless access to Azure OpenAI with RBAC

GOAL The jump host calls Azure OpenAI with its managed identity, authorized by a least-privilege role.

PART 4

How RBAC and managed identity tokens work

Azure role-based access control joins three elements in a role assignment: a **security principal** (a user, a group, a service principal or a managed identity), a **role definition** listing the allowed operations, and a **scope**, from management group down to a single resource, inherited by everything below it. Role definitions separate *Actions*, control-plane operations such as listing keys, from *DataActions*, data-plane operations such as sending a prompt.

Cognitive Services OpenAI User contains the DataActions needed for inference and read access to deployments, but no permission to list keys or change the resource. This also explains a common surprise: the Owner role has every Action but no DataActions, so an Owner cannot call the model with Entra ID until they receive a data-plane role as well.

The token comes from the Azure Instance Metadata Service at 169.254.169.254, an endpoint served by the physical host and reachable only from inside the VM, which also requires the header `Metadata: true` so that it cannot be called by accident through a browser or a forwarded request.

When the VM asks for a token, the host authenticates to Entra ID on behalf of the VM's identity and returns an OAuth 2.0 access token; the credential behind it is managed and rotated by Azure and never exposed to the operating system. The token is a JSON Web Token. Azure OpenAI verifies its signature, checks that the audience is its own and that it has not expired, and then evaluates the role assignments of the identity named in the token.

PART 4

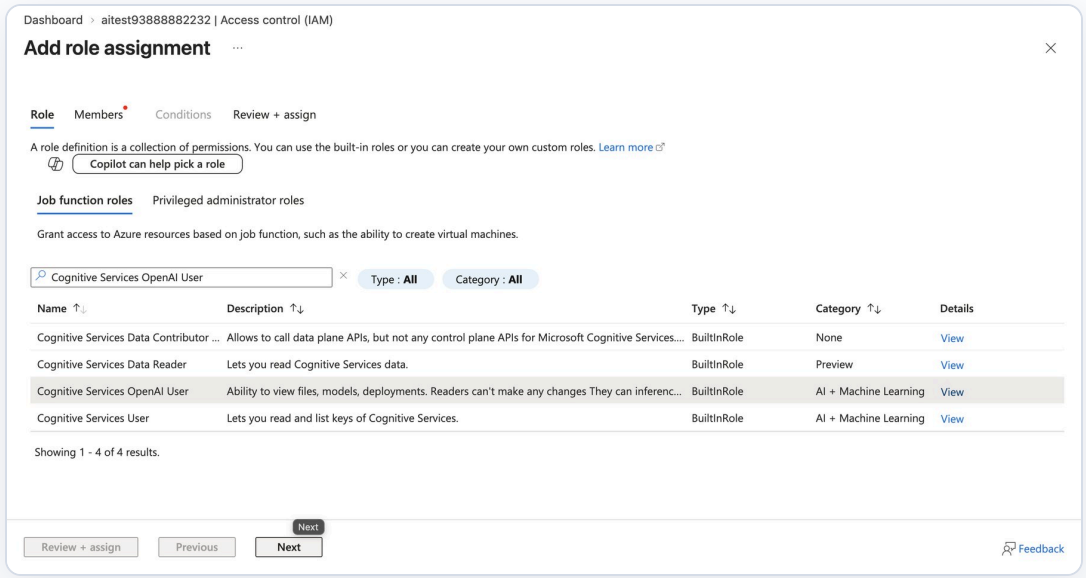
4.1 Assign the role to the managed identity

WHY With keys disabled, RBAC authorizes every call. This role allows inference only: a compromised VM cannot reconfigure the service.

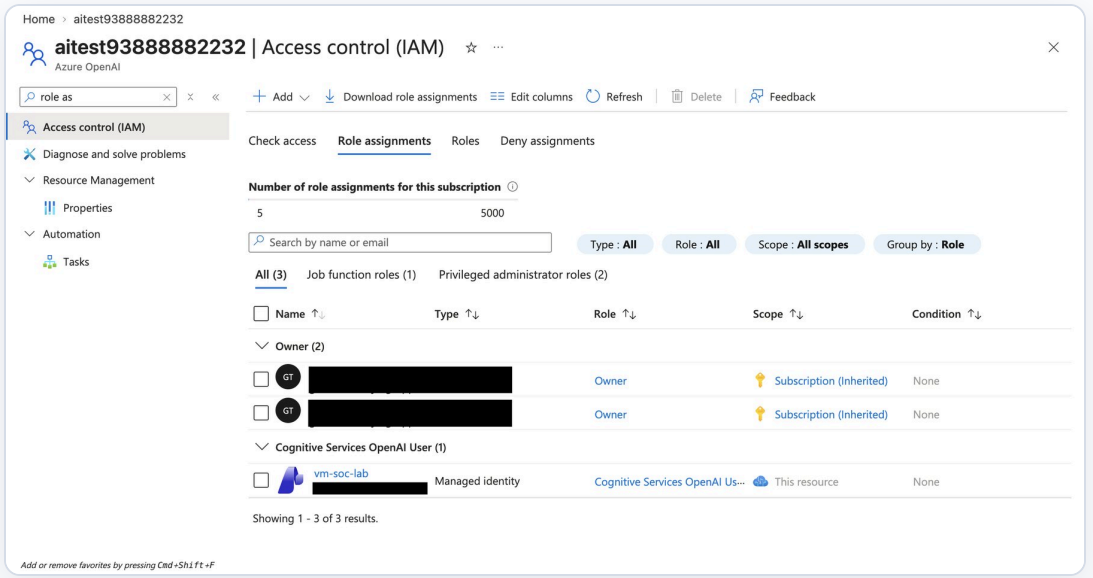
USED FOR Every call from the jump host to the model.

In `aitest93888882232`, Access control (IAM), add a role assignment with the role **Cognitive Services OpenAI User**, choose **Managed identity** as the type of member, select the virtual machine `vm-soc-lab` and confirm with Review + assign. The assignment is scoped to this resource only, and it can take a few minutes to propagate.

```
MI=$(az vm show -g DefaultResourceGroup-CHN \
  -n vm-soc-lab --query identity.principalId -o tsv)
az role assignment create --assignee-object-id "$MI" \
  --assignee-principal-type ServicePrincipal \
  --role "Cognitive Services OpenAI User" \
  --scope "$OAI_ID"
```



The built-in role



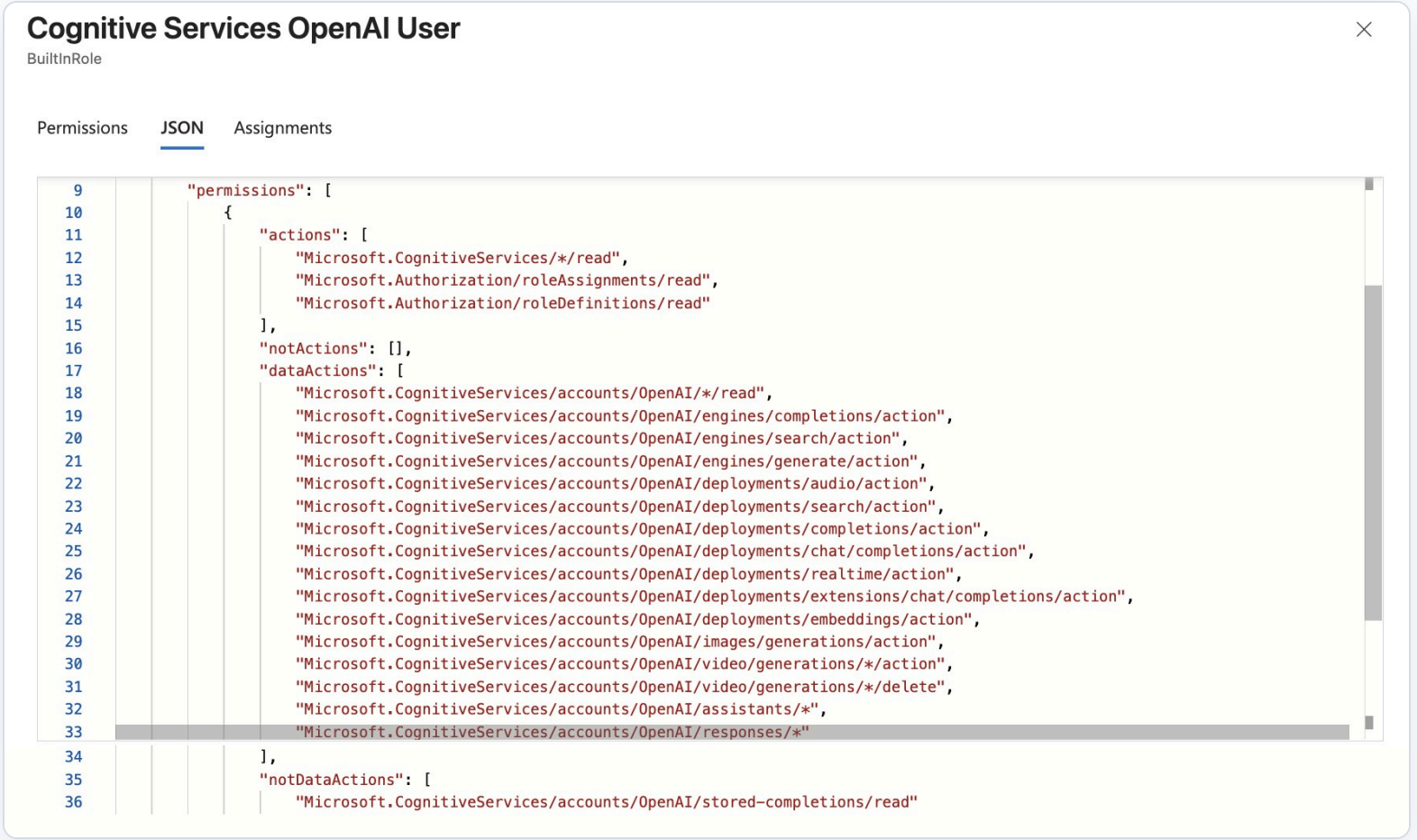
Assigned to vm-soc-lab

PART 4

4.1 What the role allows

Under Access control (IAM), Roles, the View button on Cognitive Services OpenAI User and its JSON tab show the exact permissions of the role. Its **actions**, which act on the control plane, are only read operations: the identity can see the resource and its role assignments, but it cannot change the resource, create deployments or list keys.

Its **dataActions** are the inference operations: chat completions, embeddings, images, assistants and responses. The **notDataActions** exclude reading stored completions, so a holder of this role cannot read the conversation history kept by the service.



Cognitive Services OpenAI User, JSON

PART 4

4.2 Obtain a token and inspect it

WHY Shows who issued the credential, for which service, for which identity and until when.

USED FOR The token sent in 4.3; the script obtains it the same way.

On the jump host, the first command asks the metadata service for a token for Azure AI services and stores it in a variable. Behind that single request, the Azure host recognizes which VM is asking, obtains a token from Entra ID for that VM's identity and hands it back; the VM cannot ask for a token belonging to any other identity, and no secret is written to its disk.

Decoding the payload, without ever publishing the token itself, shows the issuer (your Entra ID tenant), the audience `cognitiveservices.azure.com`, the object ID of the VM's identity and the expiry time, after which the token stops working on its own.

```
# on vm-soc-lab
URL="http://169.254.169.254/metadata/identity/oauth2/token"
URL="$URL?api-version=2018-02-01"
URL="$URL&resource=https://cognitiveservices.azure.com"
TOKEN=$(curl -s -H "Metadata:true" "$URL" | python3 -c \
  "import sys,json; print(json.load(sys.stdin)['access_token'])")
```



Decoded claims, sensitive values hidden

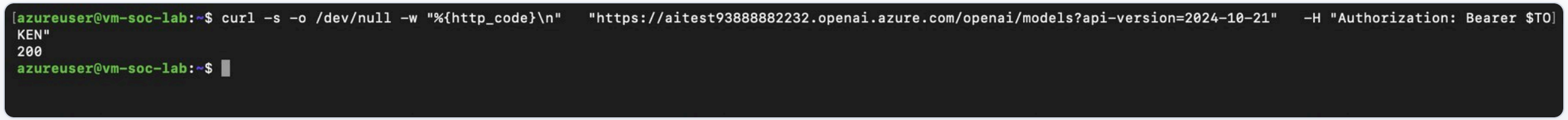
PART 4

4.3 Validate the full chain

WHY One request proves four controls: private DNS, private path, Entra ID authentication, RBAC.

USED FOR Readiness for the operator tool and the automation part.

```
# on vm-soc-lab
EP=https://aitest93888882232.openai.azure.com
curl -s -o /dev/null -w "%{http_code}\n" \
  -H "Authorization: Bearer $TOKEN" \
  "$EP/openai/models?api-version=2024-10-21"
```



The data-plane call from the jump host returns 200

Listing the available models is a lightweight data-plane call that follows the same path as a chat completion without consuming tokens. A status of **200** proves four controls in one request: the name resolved privately, the traffic used the private path, Entra ID authenticated the VM and RBAC authorized it.

A **401** means the token is missing or expired. A **403** that mentions a virtual network means the request did not use the private endpoint, so the DNS check of step 3.4 should be repeated, while a 403 with `PermissionDenied` means the role assignment is missing or has not propagated yet.

PART 5

Querying the model from inside the network

GOAL An operator tool to chat with the model and submit security events for assessment, over the same private path and identity.

PART 5

ask-openai.py

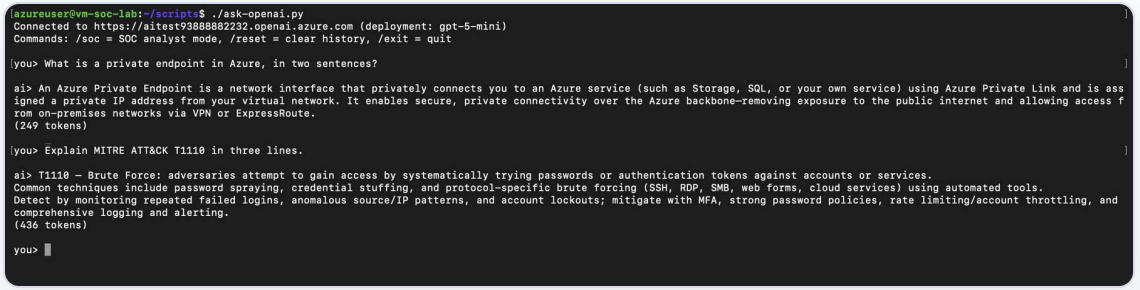
WHY The browser playground is blocked by design; analysts still need the model, over the private path only.

USED FOR Manual triage now; automatic triage in the automation part.

The Azure AI Foundry playground runs outside the private network, so it is blocked by design, yet analysts still need a way to use the model. The script `scripts/ask-openai.py` fills that gap on the jump host using only the Python standard library: before every request it obtains a token from the metadata service and calls the Chat Completions API of the deployment through the private endpoint.

```
# on your computer: copy the tools to the jump host
scp -i ~/Downloads/vm-soc-lab_key.pem -r app scripts \
  azureuser@<JUMP_PUBLIC_IP>:~

# on vm-soc-lab
cd ~/scripts && chmod +x ask-openai.py simulate-attack.sh
export AOAI_ENDPOINT=https://aitest93888882232.openai.azure.com
export AOAI_DEPLOYMENT=gpt-5-mini
./ask-openai.py                       # chat; /soc = SOC mode
./ask-openai.py "Explain MITRE T1110"  # one question
```



ask-openai.py in chat mode, on the private endpoint

PART 5

How the script talks to the model

Every request the script sends carries a list of messages, each with a role. A **system** message defines how the model should behave; the **user** messages carry your questions or, in SOC mode, the log lines read from standard input; and the previous answers of the **assistant** are included as well, so the model can follow the conversation.

The Chat Completions API is stateless, which means the model remembers nothing between requests. The script therefore resends the whole history with every call, and the reset command simply empties that list. After each answer it prints the number of tokens used, which makes the cost of every assessment visible.

In **SOC analyst mode**, the system message asks the model to explain what happened, name the most likely MITRE ATT&CK technique, judge the severity, suggest the next steps and propose a KQL query for further investigation. Piping the latest fifty events of the application into the script turns it into a first-line triage assistant (step 9.4).

The model returns text and nothing else: it cannot run commands or change any resource, and every decision stays with the analyst. Because log content is attacker-influenced (a username, for instance, could contain text written to manipulate the model), its output is always read critically before anyone acts on it.

PART 5

In production: how organizations call the model

In this guide the model is called through `ask-openai.py`, written only with the Python standard library on purpose: it needs nothing installed on the jump host, and it shows every step of the keyless flow explicitly, from requesting the token to attaching it to the call.

In a real organization, applications use the official SDKs together with the Azure Identity library. Its `DefaultAzureCredential` finds the right identity automatically, the managed identity on an Azure VM or the developer's own sign-in on a workstation, so the same code runs everywhere, and the SDK handles token refresh, retries and errors.

Security automation usually needs no code at all. In Microsoft Sentinel, a playbook calls Azure OpenAI with its own managed identity every time an incident is created and writes the assessment on the incident. Because the model has no public path, the playbook must run inside the network: **Logic Apps Standard with virtual network integration**, not the Consumption plan.

Larger organizations put Azure API Management in front of Azure OpenAI, so every application goes through a single controlled entry point with authentication, token quotas per team, request logging and failover. Whatever the method, the security model stays the same: a private endpoint, an Entra ID identity instead of a key, and a least-privilege role.

PART 6

Log Analytics, Microsoft Sentinel and SOC access

GOAL A central log store with Sentinel on top, with defined retention, cost limits and role-based access for the SOC team.

PART 6

How Log Analytics and Microsoft Sentinel fit together

A Log Analytics workspace stores data in tables, each with its own schema. The **Syslog** table receives the Linux events selected by the Data Collection Rule; the **Heartbeat** table receives one record per minute from every agent, which is how the health of the collection is monitored; and **AzureDiagnostics** receives the platform logs of Azure resources, including Azure OpenAI.

The data is queried with the Kusto Query Language, a read-only language that processes rows as a pipeline of operators. The workspace keeps data in two tiers: interactive retention, fully queryable and used by detection rules, of which Sentinel-enabled workspaces include ninety days, and long-term retention, much cheaper and kept for up to twelve years.

Microsoft Sentinel turns the workspace into a SIEM and SOAR platform without adding a separate data store. Data connectors bring in data from Microsoft and third-party sources. Analytics rules run KQL on a schedule or in near real time and create alerts, and related alerts are grouped into incidents, which carry the entities involved, such as accounts, addresses and hosts, together with a severity, an owner and a status.

Hunting queries let analysts search for threats that no rule has detected, workbooks provide dashboards, and automation rules and playbooks act on incidents automatically. In the Microsoft Defender portal, Sentinel and Defender XDR incidents are handled in a single queue.

PART 6

6.1 Create the workspace · 6.2 Enable Microsoft Sentinel

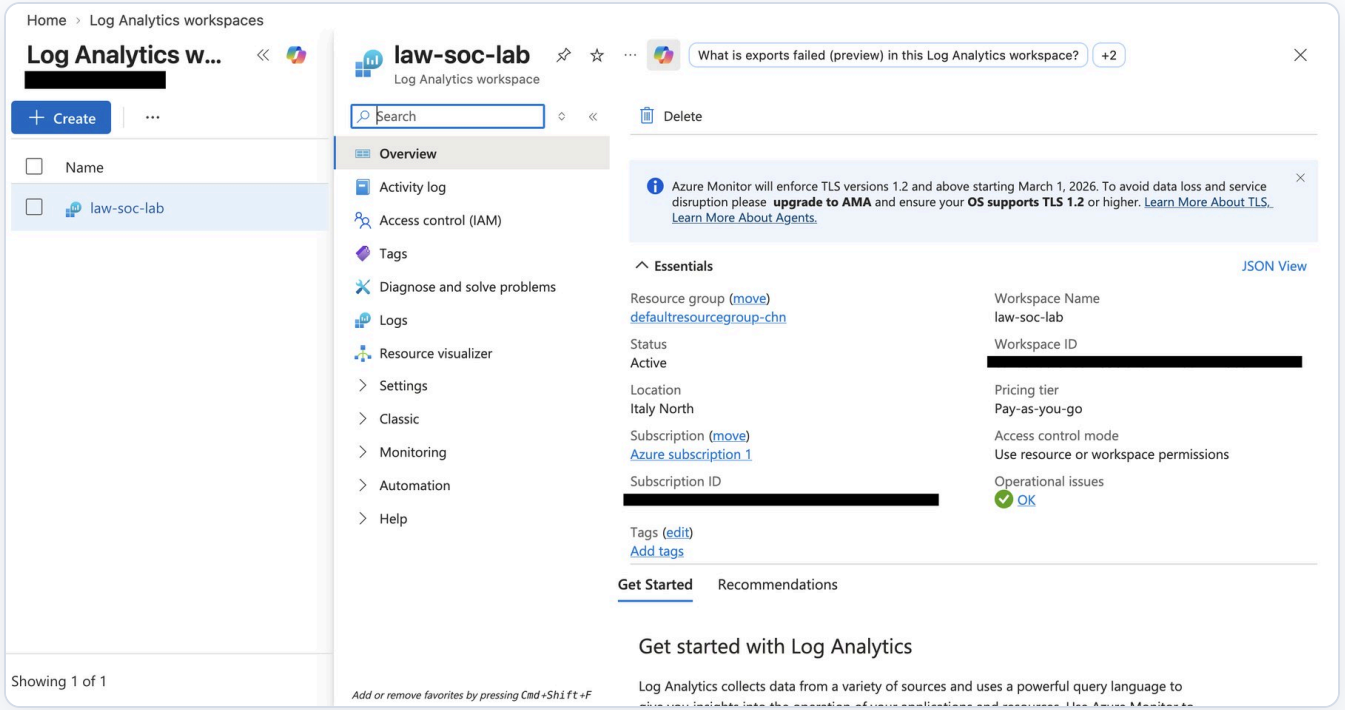
WHY Security events must be stored in one place to be searched, correlated and retained.

USED FOR VM logs (Part 8), AI audit logs (8.4), Sentinel (6.2).

Log Analytics workspaces > Create

Field	Value	Type
Name	`law-soc-lab`	Customizable
Region	Italy North: the DCR must be in the same region	Required
Pricing tier	Pay-as-you-go (commitment tiers for high volumes)	Customizable

Once the workspace exists, open **Microsoft Sentinel**, select **Create**, choose `law-soc-lab` and add it. In a company tenant, the first workspace onboarded can become the tenant's primary workspace in the Defender portal, so this step is coordinated with the security team.



law-soc-lab, captured before the tag policy had added its tags

PART 6

6.3 Retention and daily cap

WHY Retention decides how far back an investigation can look; the cap is a hard limit on ingestion cost.

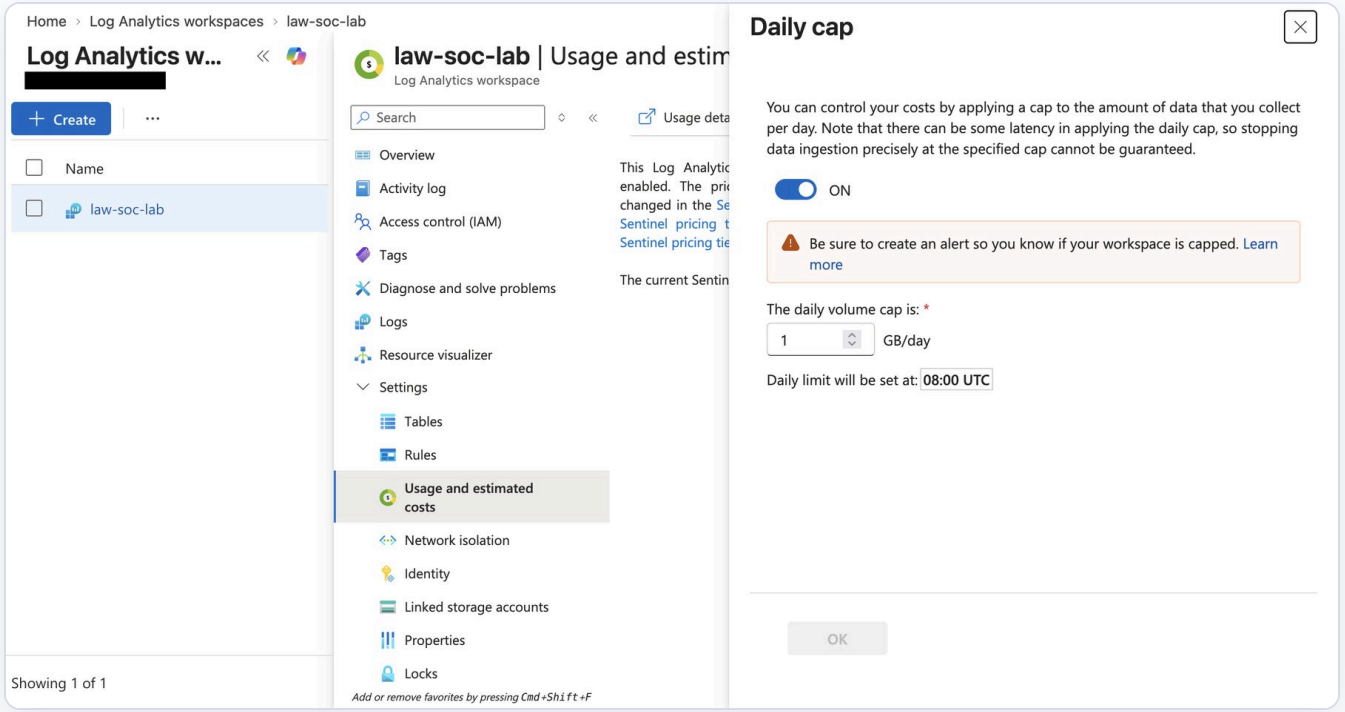
USED FOR Every investigation runs on retained data; the cap protects the budget.

```
az monitor log-analytics workspace update \
  -g DefaultResourceGroup-CHN -n law-soc-lab \
  --retention-time 90 --quota 1
```

Setting	Value	Type
Retention	90 days, the period included with Sentinel	Customizable
Daily cap	1 GB per day	Customizable

Retention decides how far back an investigation can look, and in regulated organizations it is set by the logging policy.

The daily cap, set in **Usage and estimated costs** before any data arrives, stops billable ingestion once the volume is reached. In production the cap is sized well above the expected volume and paired with an alert, because a cap that stops collection during an attack means missing evidence.



The daily cap at 1 GB/day

PART 6

How the SOC roles are structured

Microsoft Sentinel has three built-in roles that build on each other. **Reader** can view incidents, data, workbooks and rules. **Responder** can also manage incidents: assign them, change their status and severity and add comments. **Contributor** adds the ability to create and edit analytics rules, workbooks, watchlists and data connectors.

These roles map onto the jobs in a SOC: readers for auditors and management, responders for the analysts who triage incidents, and contributors for the engineers who build detections.

The roles are given to Entra ID security groups rather than to individual people. A new analyst gains access by joining the group and a leaver loses it by being removed, without anyone touching the role assignments. The same model supports access reviews and Privileged Identity Management.

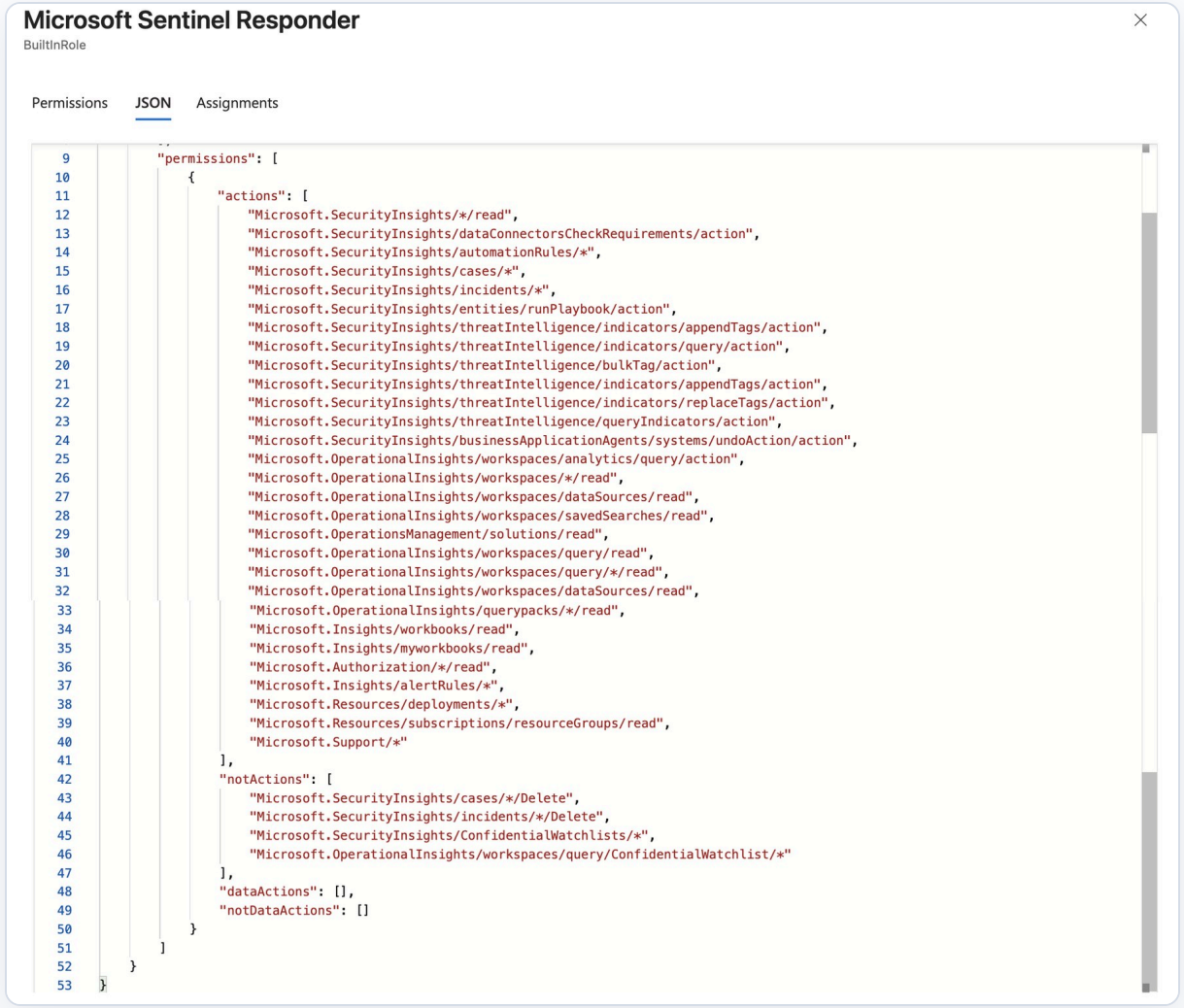
Opening a Bastion session also requires Reader on the VM, its network interface and the Bastion resource. Reader on the whole resource group would let VM administrators read the workspace and the Sentinel incidents too, so in Part 10 it is granted only on those resources.

PART 6

What the analyst role allows

The JSON of Microsoft Sentinel Responder shows least privilege applied to a SOC. Its actions give read access to all of Sentinel and to the workspace queries, full management of incidents and automation rules, and the right to run playbooks on entities.

Its notActions remove what an analyst must not do: deleting incidents and reading confidential watchlists. An analyst can work on every incident but cannot remove the evidence. The dataActions are empty, because every Sentinel permission is a control-plane operation.



Microsoft Sentinel Responder, JSON

PART 6

6.4 SOC access through Entra ID groups

WHY Access follows the job, not the person: membership handles joiners and leavers.

USED FOR Incident work, detection engineering, read-only access, VM login via Bastion.

Group	Role	Scope	Purpose
`grp-soc-analysts`	Microsoft Sentinel Responder	`law-soc-lab`	Triage and manage incidents
`grp-soc-engineers`	Microsoft Sentinel Contributor	`law-soc-lab`	Analytics rules, workbooks, connectors
`grp-soc-engineers`	Cognitive Services OpenAI User	`aitest93888882232`	Use the private model from the jump host
`grp-soc-readers`	Microsoft Sentinel Reader	`law-soc-lab`	Read-only: auditors, management
`grp-vm-admins`	Virtual Machine Administrator Login	Resource group	Entra ID login to the VMs with sudo (Part 10)
`grp-vm-admins`	Reader	VMs, NICs, Bastion	Open a Bastion session (Part 10)

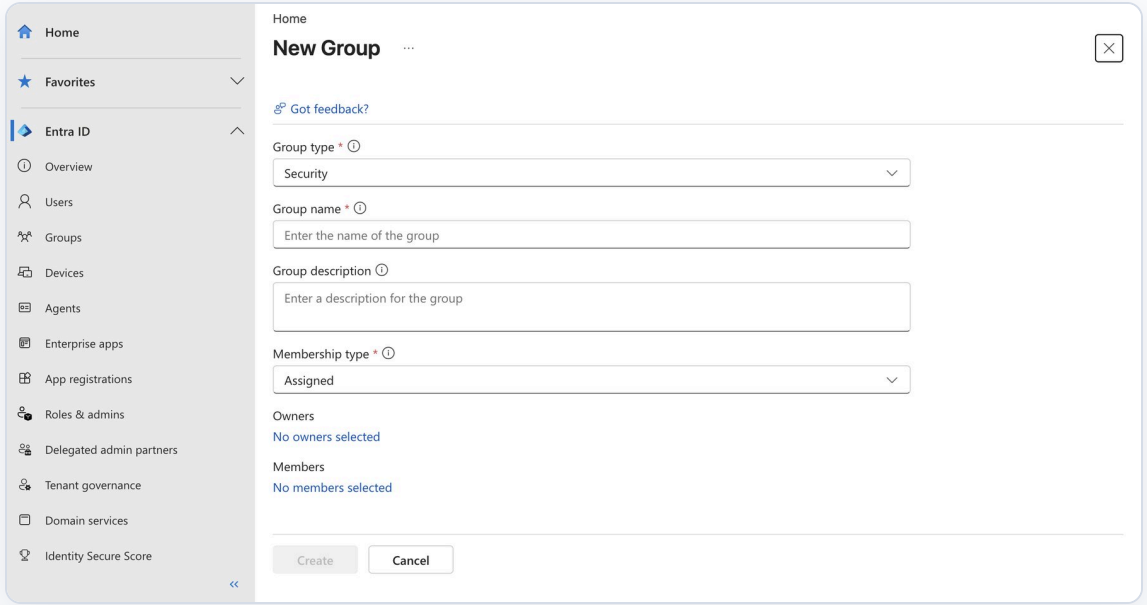
The group names can be changed (Customizable); the roles must stay as shown (Required), because the access model depends on them. Engineers use the model from the jump host, so they also need to sign in to it; the administrator is in both groups. Security defaults are enabled in Entra ID, so every sign-in behind these roles requires MFA (slide 92).

PART 6

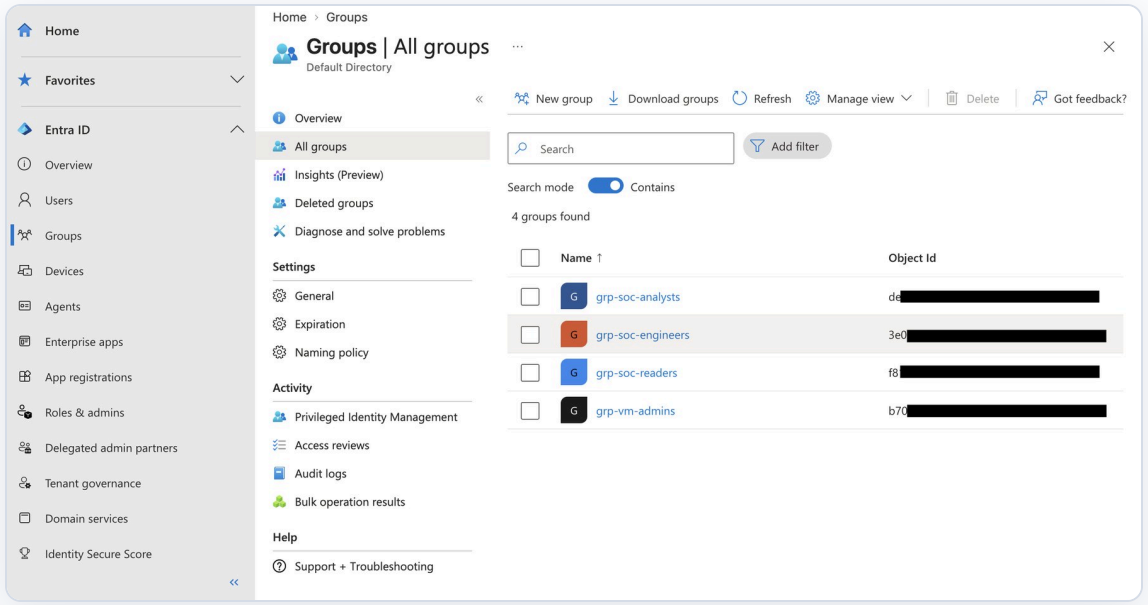
6.4 Create the groups in Microsoft Entra ID

The groups live in Microsoft Entra ID, which belongs to the tenant and does not need an Azure subscription. In the Microsoft Entra admin center, under Groups, All groups, New group, the group type is **Security**, because security groups are the type that can receive Azure roles, while Microsoft 365 groups are meant for collaboration.

The first group is `grp-soc-analysts`, with membership type **Assigned**, so members are added explicitly by an administrator; dynamic membership would require an Entra ID P1 license. The same procedure creates `grp-soc-engineers`, `grp-soc-readers` and `grp-vm-admins`.



New security group



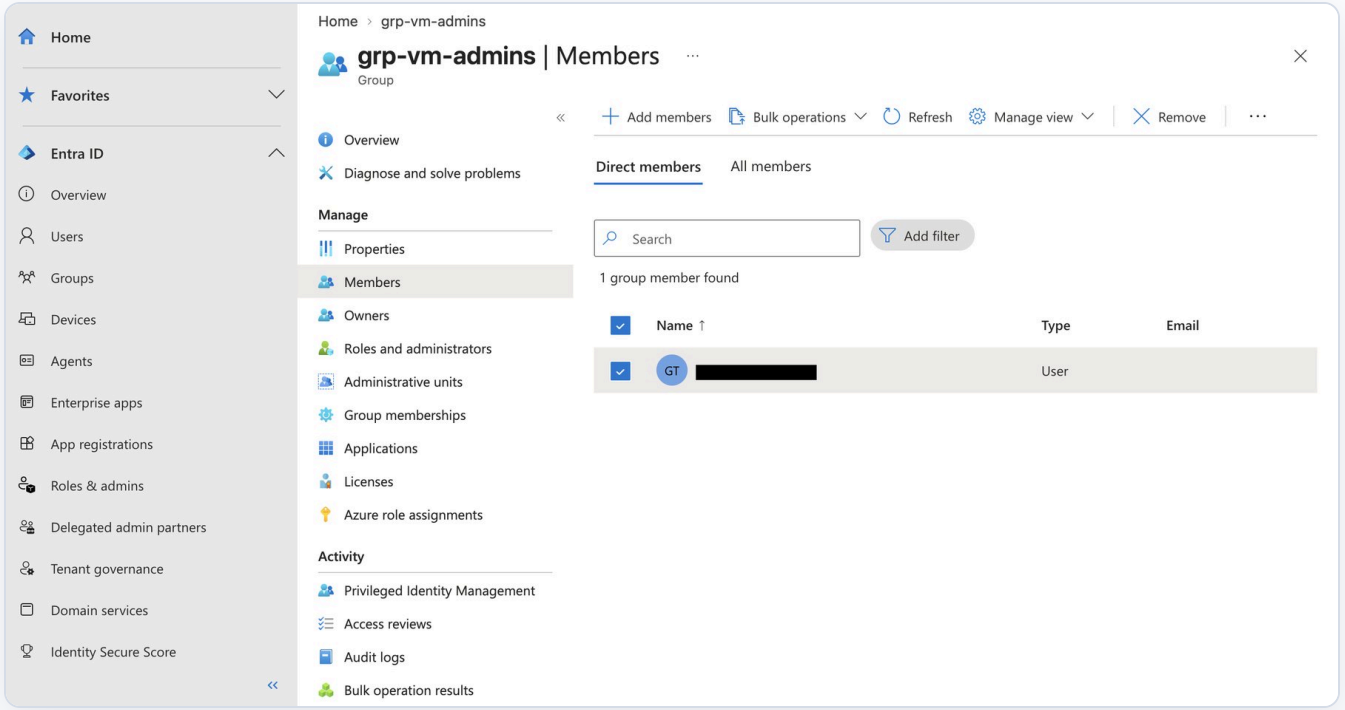
The four SOC groups

PART 6

6.4 Add the administrator to grp-vm-admins

Inside `grp-vm-admins`, the Members page starts empty. Add members opens a panel where the administrator searches for their own user, ticks it and confirms with Select, after which the page lists the user as a member of type User.

This membership is the whole access model in miniature. Nobody receives a role personally: the group holds Virtual Machine Administrator Login on the resource group and, from Part 10, Reader on the VMs, their network interfaces and Bastion. Removing the membership removes the access, without touching a single role assignment.



The administrator as the only member of grp-vm-admins

PART 6

6.4 Create the groups and assign the roles

```
RG=DefaultResourceGroup-CHN
WS_ID=$(az monitor log-analytics workspace show -g $RG -n law-soc-lab --query id -o tsv)
OAI_ID=$(az cognitiveservices account show -g $RG -n aitest93888882232 --query id -o tsv)
RG_ID=$(az group show -n $RG --query id -o tsv)
for g in grp-soc-analysts grp-soc-engineers grp-soc-readers grp-vm-admins; do
  az ad group create --display-name $g --mail-nickname $g -o none
done
gid() { az ad group show -g "$1" --query id -o tsv; }
assign() { az role assignment create --assignee-object-id "$(gid $1)" \
            --assignee-principal-type Group --role "$2" --scope "$3"; }
assign grp-soc-analysts  "Microsoft Sentinel Responder"        "$WS_ID"
assign grp-soc-engineers "Microsoft Sentinel Contributor"      "$WS_ID"
assign grp-soc-engineers "Cognitive Services OpenAI User"      "$OAI_ID"
assign grp-soc-readers   "Microsoft Sentinel Reader"           "$WS_ID"
assign grp-vm-admins     "Virtual Machine Administrator Login" "$RG_ID"
ME=$(az ad signed-in-user show --query id -o tsv)
az ad group member add --group grp-vm-admins     --member-id "$ME"
az ad group member add --group grp-soc-engineers --member-id "$ME"
# Reader for grp-vm-admins: on the VMs, NICs and Bastion only, assigned in Part 10
```

The script creates the four groups, assigns each role to its group on the right scope, and adds you to the groups you need for the rest of the guide. The result can be checked in `law-soc-lab`, Access control (IAM), Role assignments.

PART 7

The monitored workload: vm-app-lab

GOAL A realistic target to protect: a business application with no public IP that writes structured security events.

PART 7

7.1 Generate the SSH key on the jump host

WHY A key generated by Azure would be downloaded to a laptop; generated on the jump host, it never leaves the network.

USED FOR App deployment (7.3) and log reads for the AI assessment (9.4).

The target is **Acme Workplace**, a small internal portal for ordering equipment, built for this lab. It has a sign-in page, a catalog, orders, an admin console and a JSON API. Its SSH key pair is not generated by Azure: generated on the jump host instead, the private key never leaves the internal network, and only the jump host can log in to the target.

```
# on vm-soc-lab
ssh-keygen -t ed25519 -f ~/.ssh/id_app -N ""
cat ~/.ssh/id_app.pub
```

The command creates an Ed25519 key pair, an elliptic-curve algorithm that is shorter, faster and considered stronger than RSA keys of common sizes. The empty passphrase lets the jump host use the key without interaction; the key is protected by the permissions of `~/.ssh` and by the fact that only administrators can reach the jump host. The public key, a single line that starts with `ssh-ed25519`, is pasted into the wizard of the next step. Part 10 removes this key.

PART 7

7.2 Create vm-app-lab

The procedure is the same as for the jump host; these are the values that differ or that matter most.

WHY A business application must not be reachable from the internet: only inside in, only NAT out.

USED FOR Hosting the app, producing events, being the attack target.

Field	Value	Type
Name / region / zone	`vm-app-lab`, Italy North, zone 1: same zone as the jump host and the NAT Gateway	Customizable
Image / size	Ubuntu Server 24.04 LTS / Standard D2alds_v7. nginx and Flask need very few resources	Customizable
SSH public key source	Use existing public key, paste the line from 7.1. There is no password	Required
Public inbound ports	None	Required
Virtual network / subnet	`vnet01` / `subnet-1`	Required
Public IP	None: outbound uses the NAT Gateway (2.4)	Required
System assigned managed identity	Enabled: the Azure Monitor Agent authenticates with it	Required
Alerts / OS guest diagnostics	Off: logs are collected by the DCR in Part 8	Required

PART 7

7.2 Validate and restrict access to the target

On the Overview page of `vm-app-lab`, the public IP field must be empty and the virtual network must read `vnet01/subnet-1`. The private address shown there, **172.16.0.5** in this project, is `<APP_PRIVATE_IP>` in the following commands.

```
# on vm-soc-lab
ssh -i ~/.ssh/id_app azureuser@<APP_PRIVATE_IP>
# on vm-app-lab: must print the public IP of the NAT Gateway
curl -s ifconfig.me; echo
```

The jump host reaches the target with the internal key, and the target's first outbound request shows the NAT Gateway's address, confirming both directions of the design.

By default `AllowVnetInBound` would let every machine in the hub and the spoke reach every port of the target. Its NSG therefore gets three explicit rules:

Priority	Source	Ports	Action
100	Jump host private IP /32	22, 80	Allow
110	`10.0.2.0/26` (Bastion subnet)	22	Allow
4000	Service tag `VirtualNetwork`	Any	Deny

The target now accepts SSH and HTTP only from the jump host and SSH from Bastion, and refuses the rest of the virtual network; the internet was already refused by the default rules.

PART 7

7.3 Deploy the application

WHY A realistic workload with production components generates real security events.

USED FOR The data Sentinel collects, the model assesses and the attack targets.

The `app` folder was copied to the jump host in Part 5. From there it travels to the target, where the installer runs:

```
# on vm-soc-lab
scp -i ~/.ssh/id_app -r ~/app azureuser@<APP_PRIVATE_IP>:~
ssh -i ~/.ssh/id_app azureuser@<APP_PRIVATE_IP>
# on vm-app-lab
cd ~/app && cp deploy/* . && ./install.sh
systemctl status acme nginx --no-pager \
  | grep -E "acme|nginx|Active"
curl -s localhost/health; echo
```

The deployment is complete when both services report **active (running)** and the health check answers with status ok.

The application runs as the three-layer stack used by many production Python services. **nginx** listens on port 80 as a reverse proxy, forwards each request to the application server on 127.0.0.1:5000, adds the client address in the X-Forwarded-For header so the application logs the real source, and writes its own access log to syslog.

gunicorn runs two worker processes of the Flask application and listens only on localhost, so it is never reachable from the network. **systemd** manages it as the service `acme`: it starts it at boot, restarts it if it fails, runs it as the unprivileged user `www-data`, and loads the session secret from `/etc/acme.env`, readable only by root, so no secret lives in the code.

PART 7

7.4 Security events written by the application

The application writes to the `auth` syslog facility with the tag `acme-app`, one key-value event per line:
`EVENT user='name' ip=1.2.3.4 key='value'`

WHY A SIEM can only detect what the application records; structured events parse cleanly in KQL.

USED FOR The queries of Part 9 and the detection rule of the automation part.

Event	Trigger
`LOGIN_SUCCESS` / `LOGIN_FAILED`	Sign-in attempts, with reason='bad_password' or 'unknown_user'
`LOGOUT`, `REGISTER`, `PASSWORD_CHANGED`	Account lifecycle and credential changes
`AUTH_REQUIRED`, `API_UNAUTHORIZED`	Protected pages or API calls without a session
`PRIVILEGE_DENIED`, `ACCESS_DENIED`	Admin console or another user's order (IDOR attempt)
`ADMIN_ACCESS`, `ORDER_CREATED`	Administrative and business activity
`SUSPICIOUS_INPUT`	SQL injection, XSS, path traversal or command injection patterns
`APP_ERROR`	Unhandled server errors

Suspicious input is recorded rather than blocked so the signal reaches the SIEM, while the application still uses parameterized SQL, salted password hashes, constant-time credential checks and security headers. Values are written as they arrive, so a username containing a quote and a fake `ip=` field could forge the parsed source address; production logs JSON and parses it with `parse_json()`.

PART 7

7.5 Access the portal without exposing it

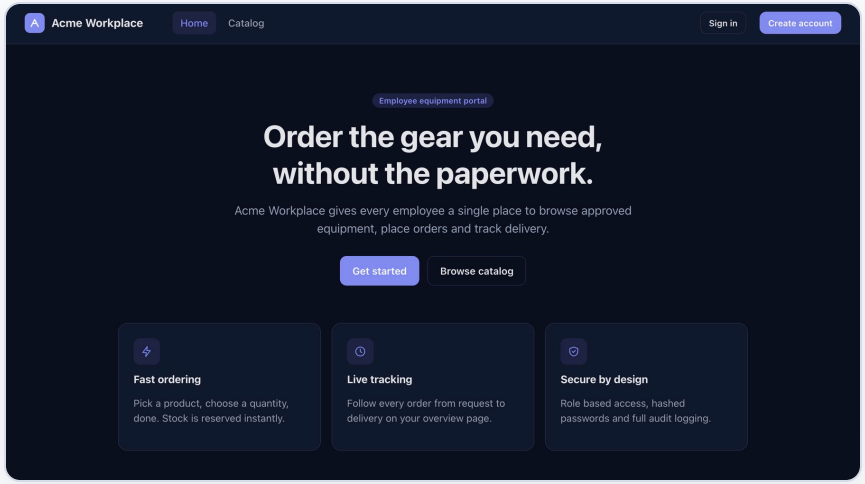
WHY The portal has no public endpoint by design; the tunnel gives browser access without opening any port.

USED FOR Testing the app; replaced by a Bastion tunnel in Part 10.

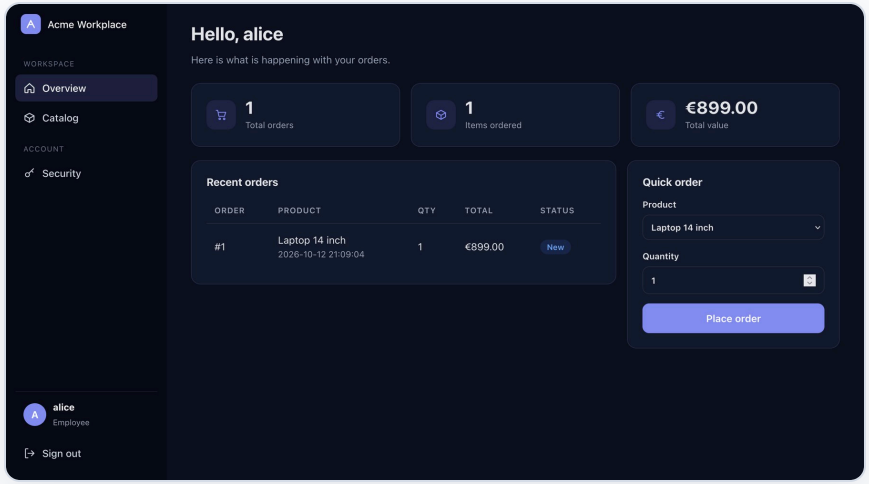
```
# on your computer
ssh -N -i ~/Downloads/vm-soc-lab_key.pem \
  -L 8080:<APP_PRIVATE_IP>:80 \
  azureuser@<JUMP_PUBLIC_IP>
```

SSH local port forwarding opens port 8080 on your computer, carries every connection to it inside the encrypted session to the jump host, and the jump host opens a new connection to port 80 of the target on the private network.

The `-N` option opens the tunnel without a remote shell, so the terminal stays silent. With the tunnel open, `http://localhost:8080` shows the portal, where the test accounts `alice`, `bob` and `admin` are available.



Landing page

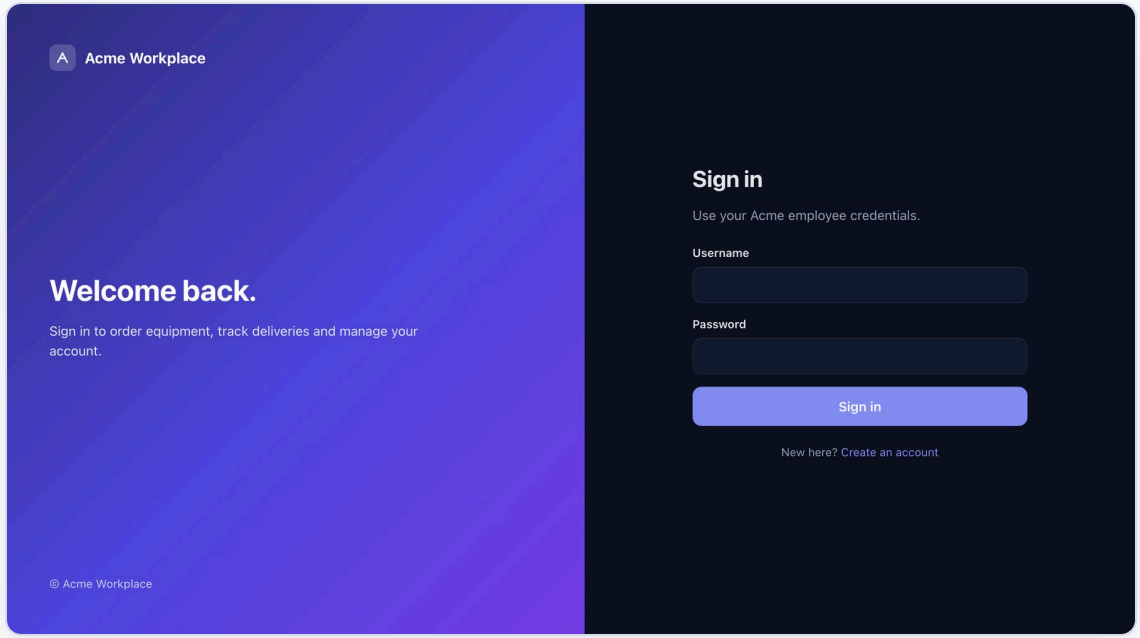


Employee overview

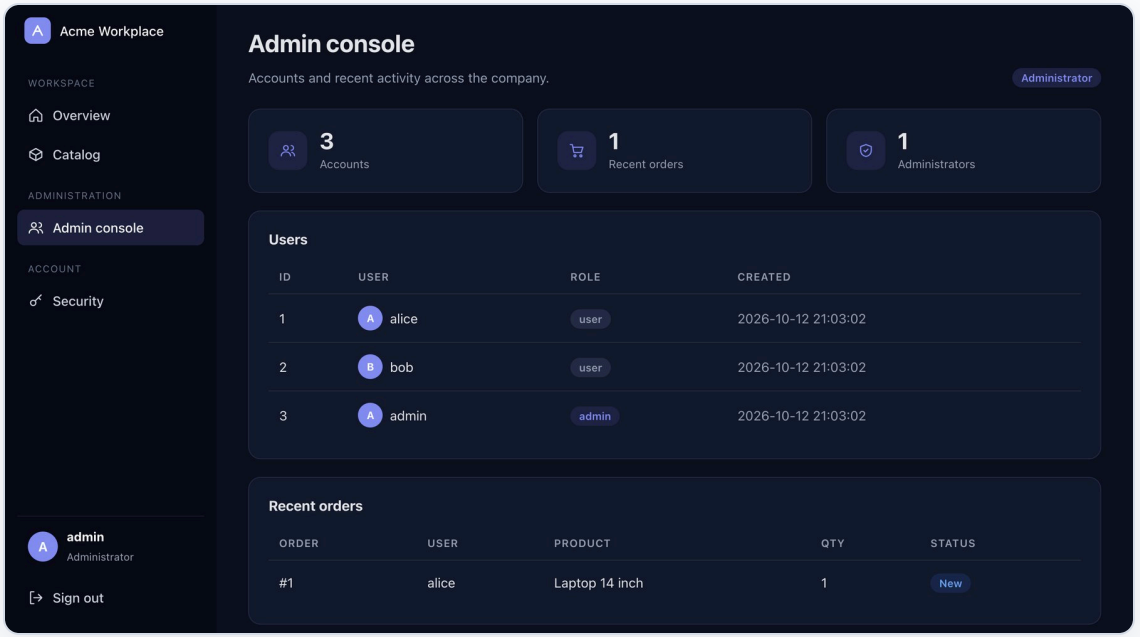
PART 7

The application surface

Authentication, pages that depend on the user's role and an administration console: enough to produce the events a SOC actually watches.



Sign-in



Admin console

PART 8

Log collection with the Azure Monitor Agent

GOAL Both VMs send their security logs, and Azure OpenAI sends its audit logs, to law-soc-lab.

PART 8

How the Azure Monitor Agent and the DCR work

Data collection in Azure Monitor has three parts. The **Azure Monitor Agent** is the collector installed on each VM as the extension `AzureMonitorLinuxAgent`; it replaces the legacy Log Analytics agent, retired in August 2024. The **Data Collection Rule** is an Azure resource that describes the data sources (here Linux syslog with selected facilities and levels), optional KQL transformations that can drop or reshape data before it is stored and billed, and the destinations (here `law-soc-lab`). The **association** links a rule to a VM.

When a VM is associated, the agent authenticates to Azure Monitor with the VM's managed identity, downloads the rules that apply to it, configures rsyslog to forward the selected facilities and sends the data over HTTPS through the NAT Gateway.

Syslog severities are ordered from the most severe, emergency, through alert, critical, error, warning, notice and informational, down to debug. The minimum level chosen in the rule is a threshold: the agent collects that level and every more severe one, so a minimum of info collects everything except debug, while a minimum of warning keeps only warnings and above. Each facility has its own threshold.

Azure resources keep their platform logs only when a diagnostic setting routes them somewhere. For Azure OpenAI the categories are Audit, RequestResponse (one record per API request with the operation, the caller, the status and the duration) and Trace. These records describe the requests; they do not contain the prompts or the completions.

PART 8

8.1 Create the Data Collection Rule

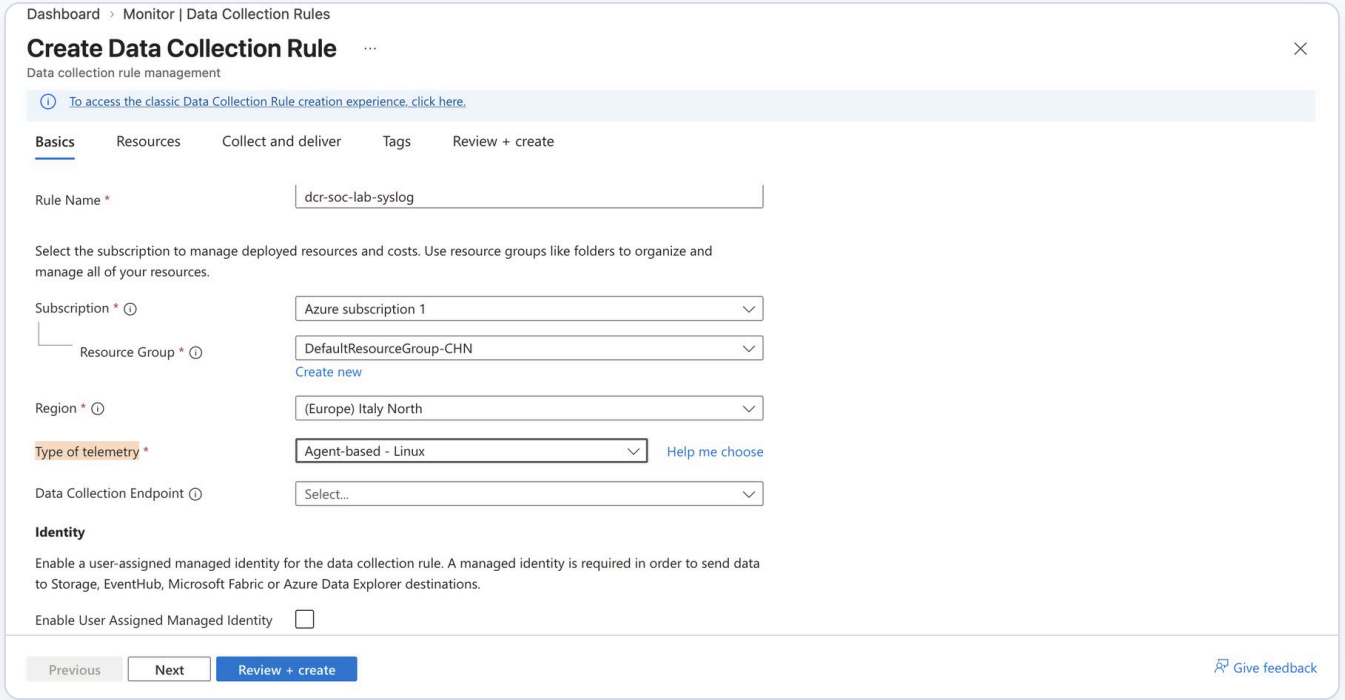
WHY Logs left on a server are lost if it is compromised; the DCR defines what leaves the servers and where it goes.

USED FOR Every syslog event in the workspace.

Monitor > Data Collection Rules > Create

Field	Value	Type
Rule name	`dcr-soc-lab-syslog`	Customizable
Region	Italy North, same as the workspace	Required
Type of telemetry	Agent-based – Linux	Required
Data Collection Endpoint	None: not needed here	Required
User-assigned identity	Disabled: only for Storage, Event Hubs, Fabric	Required

The rule is the single place that decides which logs leave the servers, at which severity, and where they go.



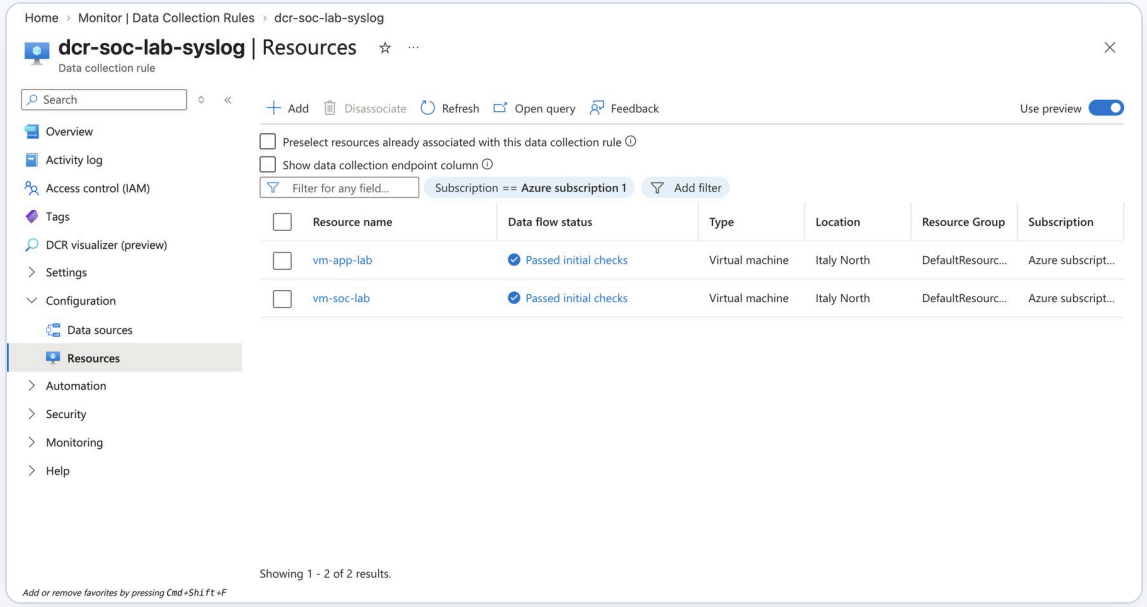
Basics

PART 8

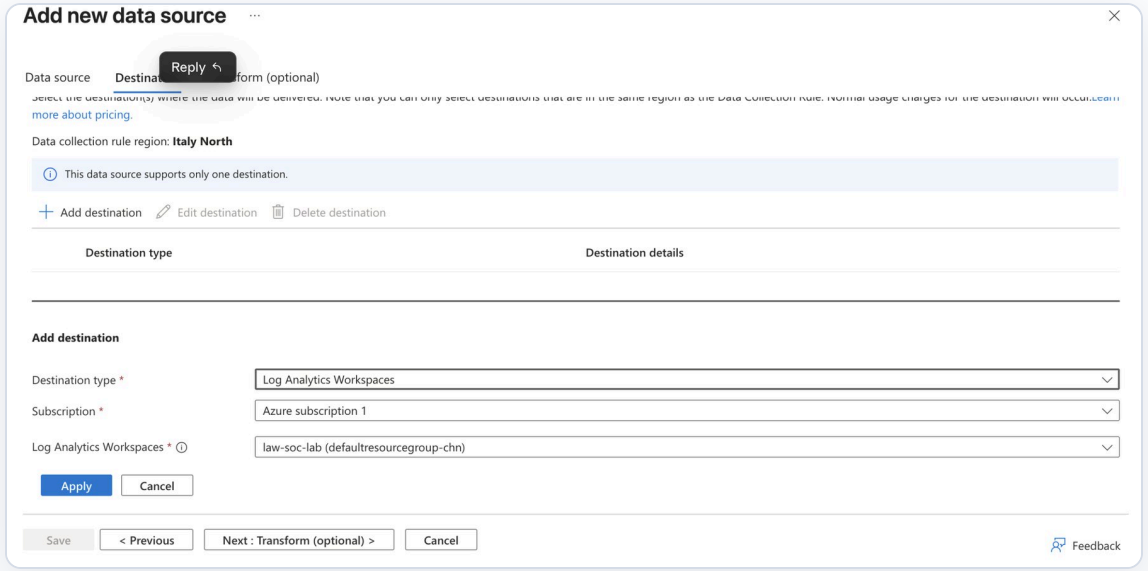
8.1 Resources and destination

On the **Resources** tab, add both `vm-soc-lab` and `vm-app-lab`. The association installs the Azure Monitor Agent automatically on each machine, where it authenticates with the VM's managed identity and reaches Azure Monitor through the NAT Gateway.

On **Collect and deliver**, add a data source of type Linux Syslog with the facilities and levels described on the next slide, and choose Azure Monitor Logs with the workspace `law-soc-lab` as its destination. Review + create completes the rule.



Both VMs associated



Destination law-soc-lab

PART 8

8.2 Syslog facilities: required set and enterprise baseline

WHY Everything is costly and noisy, too little leaves blind spots: each facility carries events a SOC uses.

USED FOR Required set for detections; baseline for persistence and tampering investigations.

Facility	Min. level	Purpose	Type
`LOG_AUTH`	`LOG_INFO`	Acme application security events	Required
`LOG_AUTHPRIV`	`LOG_INFO`	SSH logins, sudo, su	Required
`LOG_LOCAL7`	`LOG_INFO`	nginx access logs	Required
`LOG_CRON`	`LOG_WARNING`	Malicious scheduled jobs for persistence (T1053.003)	Customizable
`LOG_DAEMON`	`LOG_WARNING`	New services for persistence (T1543.002), security tools stopped	Customizable
`LOG_KERN`	`LOG_WARNING`	Unexpected kernel modules, packet filter drops	Customizable
`LOG_SYSLOG`, `LOG_USER`	`LOG_WARNING`	Logging subsystem failures, generic process errors	Customizable

The first three facilities are what this project's queries and detections depend on, so they are collected in full. The others form an enterprise baseline that supports investigations of persistence and tampering, limited to warnings and above to keep routine noise out. In a mature SOC, every data source is there because a detection or an investigation needs it.

PART 8

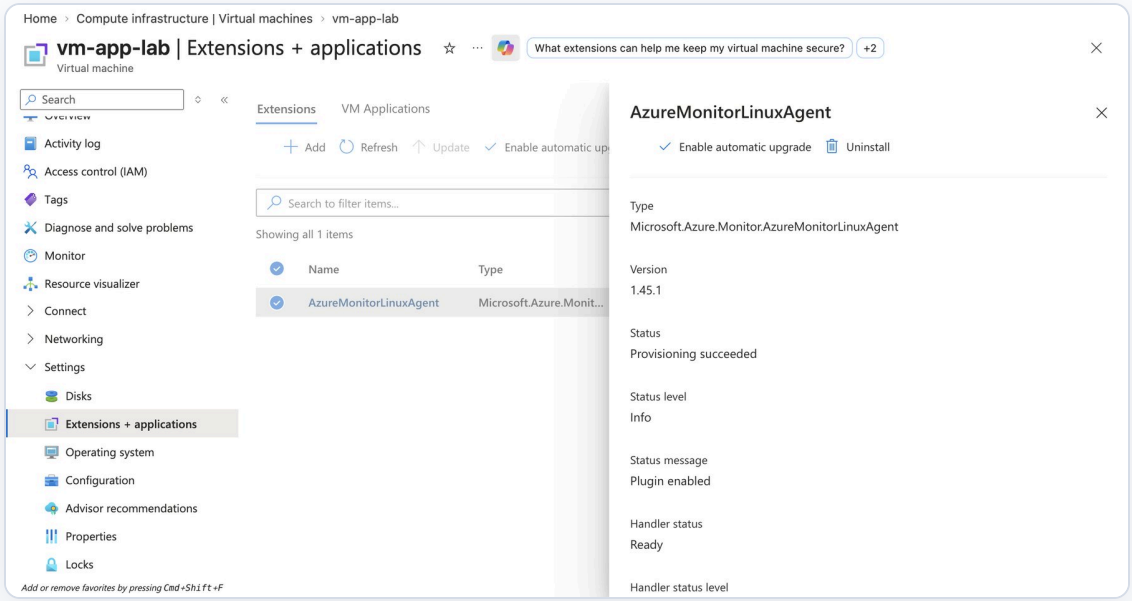
8.3 Verify the agent

WHY A stopped agent means silent gaps, often noticed only during an incident.

USED FOR Confirmation before the end-to-end test.

On each VM, Settings, **Extensions + applications** must list `AzureMonitorLinuxAgent` with the status **Provisioning succeeded**, and on the machine itself the service must be active and running.

```
# on vm-app-lab
systemctl status azuremonitoragent --no-pager | head -5
```



Extension provisioned

From the workspace, the `Heartbeat` table shows the last record received from every machine; a machine whose heartbeat is more than a few minutes old has stopped sending data. In production that query becomes an alert.



Agent running on vm-app-lab

PART 8

8.4 Send the Azure OpenAI audit logs to the workspace

WHY The AI service is itself a resource to monitor: who called the model, when, with which result.

USED FOR Auditing model use in the same workspace as the other security data.

```
RG=DefaultResourceGroup-CHN
OAI_ID=$(az cognitiveservices account show -g $RG -n aitest93888882232 --query id -o tsv)
WS_ID=$(az monitor log-analytics workspace show -g $RG -n law-soc-lab --query id -o tsv)
az monitor diagnostic-settings create --name diag-openai-to-law \
  --resource "$OAI_ID" --workspace "$WS_ID" \
  --logs '[{"categoryGroup":"allLogs","enabled":true}]' \
  --metrics '[{"category":"AllMetrics","enabled":true}]'
```

The diagnostic setting routes every log category of the resource, including any category Microsoft adds in the future, and its metrics to the same workspace as the other security data. After a few calls from the jump host, the records appear in the `AzureDiagnostics` table, where the resource provider, the operation name and the result signature identify each request; a spike of calls, unexpected status codes or calls outside working hours become searchable and alertable like any other event.

```
AzureDiagnostics
| where ResourceProvider == "MICROSOFT.COGNITIVESERVICES"
| summarize Calls = count() by OperationName, ResultSignature
```

PART 9

Telemetry validation and attack simulation

GOAL Prove that events travel end to end from the application to Microsoft Sentinel, then generate a realistic attack, observe it and ask the model to assess it.

PART 9

9.1 Pipeline check

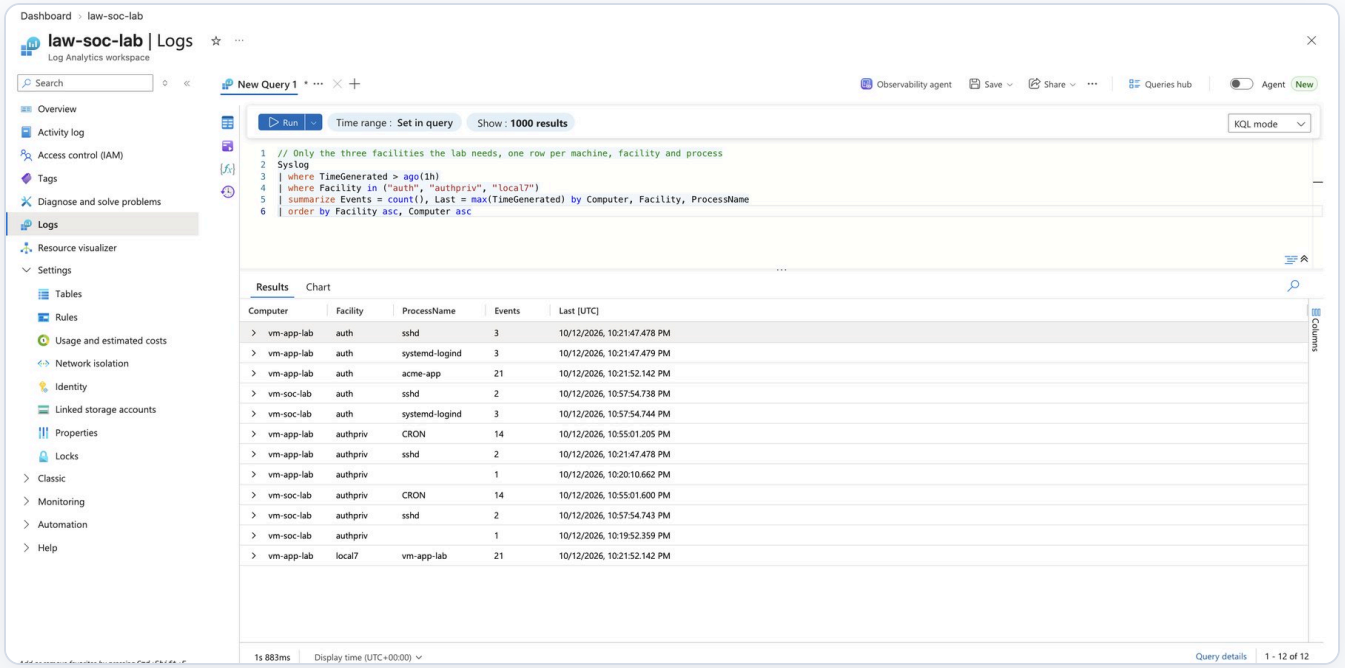
WHY Before relying on the pipeline for detection, every source must be proven to send data.

USED FOR A reusable check whenever a server is added.

```
// kql/03-pipeline-check.kql
Syslog
| where TimeGenerated > ago(1h)
| where Facility in ("auth", "authpriv", "local7")
| summarize Events = count(),
            Last = max(TimeGenerated)
    by Computer, Facility, ProcessName
| order by Facility asc, Computer asc
```

In `law-soc-lab`, Logs, in KQL mode, this query keeps the last hour first, which is the most efficient way to filter, keeps the three required facilities, then counts the events and finds the most recent one for each machine, facility and process.

The result must contain `auth` and `authpriv` rows from both VMs, and `local7` rows from `vm-app-lab` only, because nginx runs only there. The first events can take five to ten minutes to appear.



In the lab, nginx's syslog tag was parsed as the host name (ProcessName vm-app-lab)

PART 9

9.2 Simulate an attack

WHY Attack visibility must be tested, not assumed; a repeatable simulation produces known events.

USED FOR Queries in 9.3, the AI assessment in 9.4 and the detection rule.

```
# on vm-soc-lab
cd ~/scripts && ./simulate-attack.sh <APP_PRIVATE_IP>
```

The jump host plays the attacker and runs `scripts/simulate-attack.sh` against the portal. The script starts with password guessing, sending twelve sign-in attempts with different passwords against the `admin` account, MITRE ATT&CK technique **T1110.001**; each attempt produces a `LOGIN_FAILED` event with the reason `bad_password`. It then tries common usernames such as root, administrator, test, guest and support, and each produces `LOGIN_FAILED` with the reason `unknown_user`.

Next it probes the catalog with a SQL injection, a cross-site scripting and a path traversal payload, which the application records as `SUSPICIOUS_INPUT`, and finally it calls the orders API without a session, recorded as `API_UNAUTHORIZED`. The script prints the HTTP status of every request, and because it runs inside the network it also models a compromised internal machine attacking another one.

> Running the attack from the jump host is a lab shortcut: the attacker's address is also the administrator's, so any automatic block must exclude it. A production test runs from a separate attacker machine.

PART 9

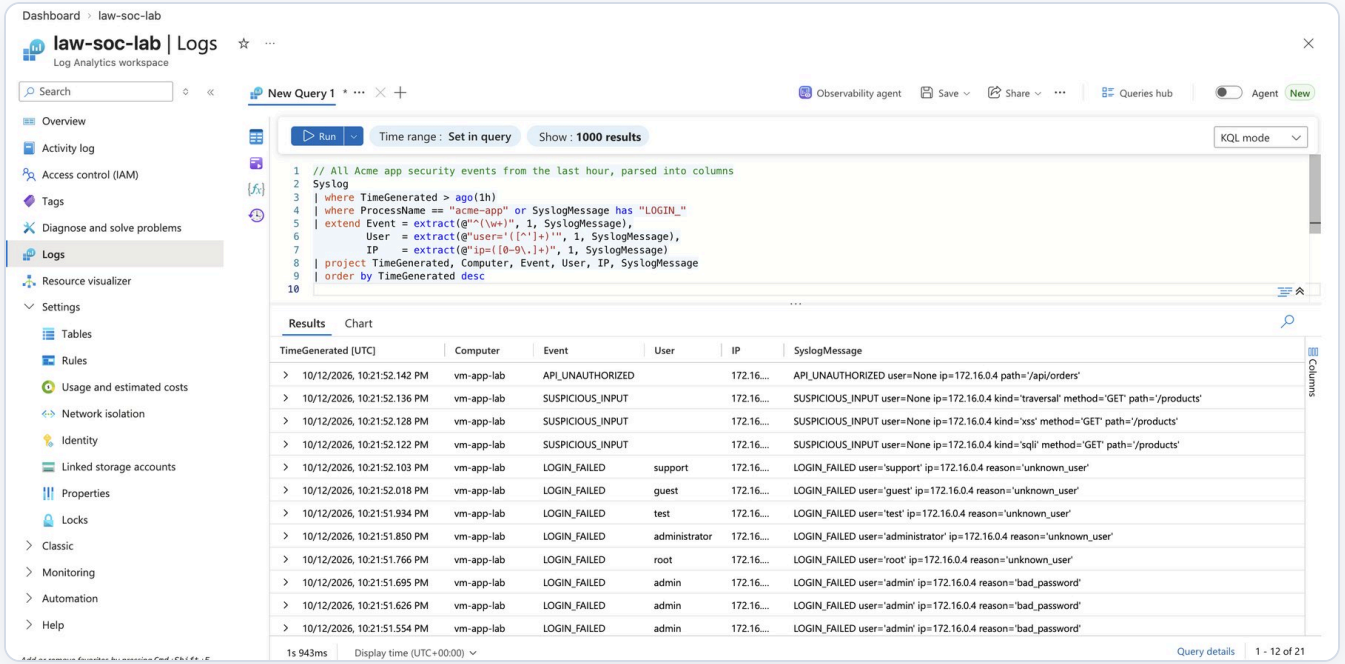
9.3 Observe the attack in Log Analytics

WHY Raw lines become columns (event, user, IP): exactly what an analyst and a rule need.

USED FOR The parsing reused by the brute-force detection rule.

```
Syslog
| where TimeGenerated > ago(1h)
| where ProcessName == "acme-app"
| extend
    Event = extract(@"^(\w+)", 1, SyslogMessage),
    User  = extract(@"user='([^']+)'", 1, SyslogMessage),
    IP    = extract(@"ip=([0-9\.]+)", 1, SyslogMessage)
| project TimeGenerated, Computer, Event, User,
          IP, SyslogMessage
| order by TimeGenerated desc
```

The `extend` operator adds calculated columns to every row, and `extract` applies a regular expression to the message: the first word becomes the event type, the text after `user` becomes the account and the address after `ip` becomes the source. The source shown is the jump host's private address, **172.16.0.4**, because the attack runs inside the network, and that is exactly the field a brute-force rule groups on.



The attack parsed into columns. The query in the screenshot also matched or SyslogMessage has "LOGIN_", which the query on the left drops

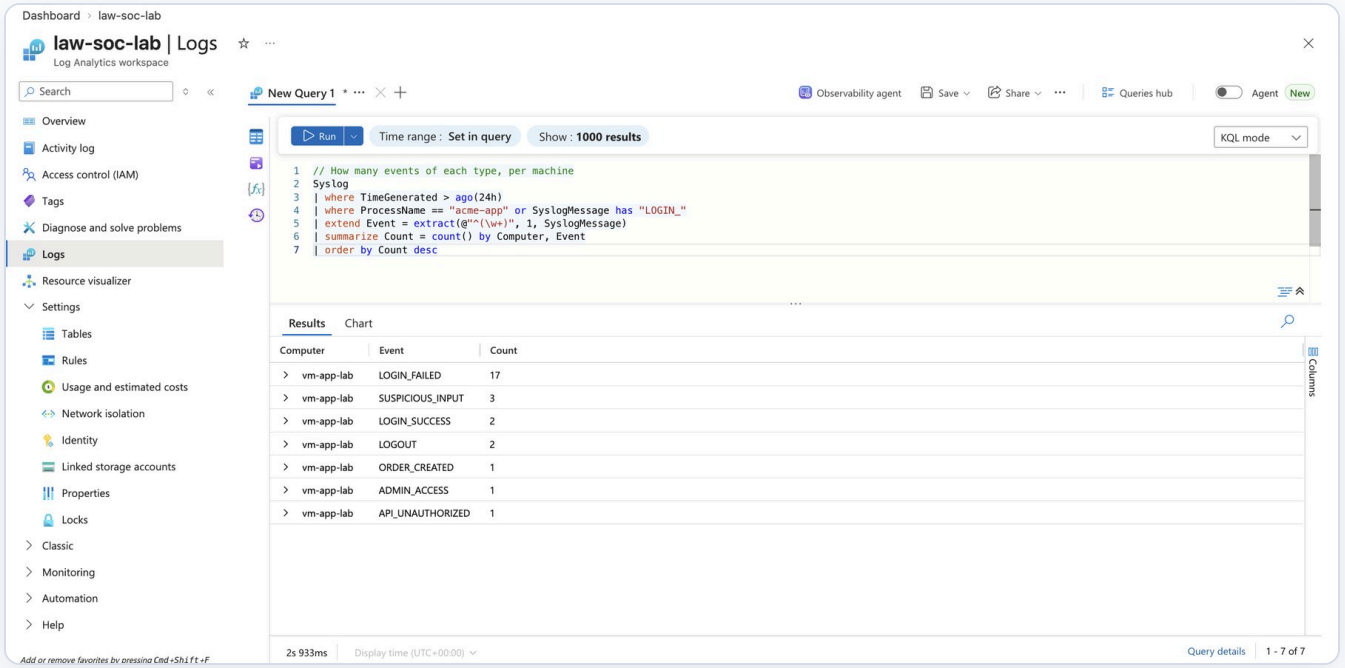
PART 9

9.3 The attack pattern at a glance

```
Syslog
| where TimeGenerated > ago(24h)
| where ProcessName == "acme-app"
| extend Event = extract(@"^(\w+)", 1, SyslogMessage)
| summarize Count = count() by Computer, Event
| order by Count desc
```

Summarizing by event type makes the pattern obvious: **17 failed sign-ins**, **3 suspicious inputs** and **1 unauthorized API call** from a single source, against only two successful sign-ins. The Chart tab of Log Analytics turns the same result into a bar chart.

Parsing at query time needs no change to ingestion. In production a parser used this often is saved as a KQL function, so every rule and every analyst relies on the same logic.



Events by type over 24 hours

ILLUSTRATION

9.4 Ask the model to assess the events

WHY Triage of the attack in plain language, without leaving the private network.

USED FOR Manual triage now; the automated playbook in the design part.

```
# on vm-soc-lab
ssh -i ~/.ssh/id_app \
  azureuser@<APP_PRIVATE_IP> \
  "sudo journalctl -t acme-app \
   -n 50 --no-pager" \
  | ./ask-openai.py --soc \
    "Assess these events"
```

The analyst pipes the latest application events into `ask-openai.py` in SOC analyst mode. The model explains what happened, names the most likely ATT&CK technique, judges the severity and proposes next steps and a KQL query.

The model returns text only, and the analyst decides. The command uses the internal key of 7.1, so it works only until Part 10 removes it; in production the events come from Log Analytics.

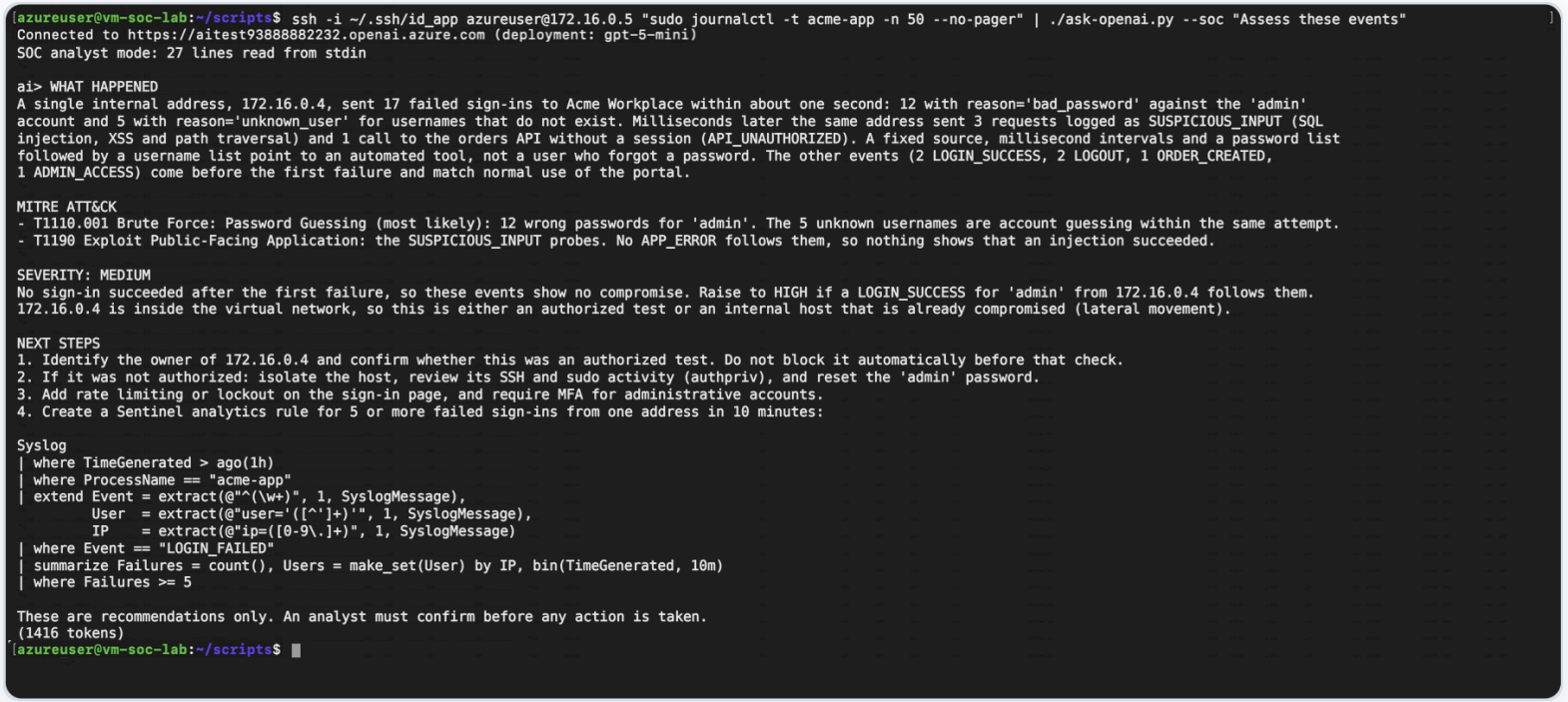


Illustration: the kind of assessment SOC mode returns for the events of 9.3. The wording varies between runs.

PART 10

Secure administrative access: Azure Bastion and Microsoft Entra ID

GOAL Remove every public entry point. Administrators connect through Bastion with their Entra ID identity; the jump host loses its public IP.

PART 10

How Azure Bastion and Entra ID login work

Azure Bastion is a managed service that Azure runs and patches inside the dedicated subnet `AzureBastionSubnet`. A session begins when the administrator connects to the Bastion endpoint over TLS on port 443, from the portal or the Azure CLI, and Azure checks both the Entra ID sign-in and the Reader permission on the VM, its network interface and the Bastion resource. Bastion then opens an SSH connection to the private address of the VM through the hub and the peering, so the VM needs neither a public IP nor an inbound internet rule.

The Developer tier is free but limited to the portal and to a single network. Basic adds connections to peered networks, Standard adds the native client, tunneling and scaling used in this project, and Premium adds session recording and private-only deployment.

Entra ID login is provided by the `AADSSHLoginForLinux` extension. When an administrator connects with Entra ID authentication, the Azure CLI uses their existing Entra ID session, already protected by MFA, to obtain a short-lived OpenSSH certificate. The VM validates the certificate and checks whether the user holds Virtual Machine Administrator Login, which grants sudo, or Virtual Machine User Login, and on the first login it creates a local account named after the user principal name.

No long-lived credential exists anywhere in this flow: revoking the role or disabling the account ends access immediately.

PART 10

10.1 Deploy Azure Bastion in the hub

WHY No public IP on the VMs and no exposed port 22: access requires an authenticated Azure session.

USED FOR Every administrative session and the portal tunnel.

Bastion is deployed in the hub, where it can reach the VMs of the peered spoke, inside the subnet whose name `AzureBastionSubnet` and minimum size of /26 are both mandatory.

Setting	Value	Type
Tier	Standard: the native client and tunneling of step 10.4 need it	Customizable
Native client support	Enabled at creation	Required
Subnet	`AzureBastionSubnet`, `10.0.2.0/26`	Required

The public address created here belongs to the managed service, not to any VM, and accepts only TLS connections from authenticated Azure users. Deployment takes about ten minutes.

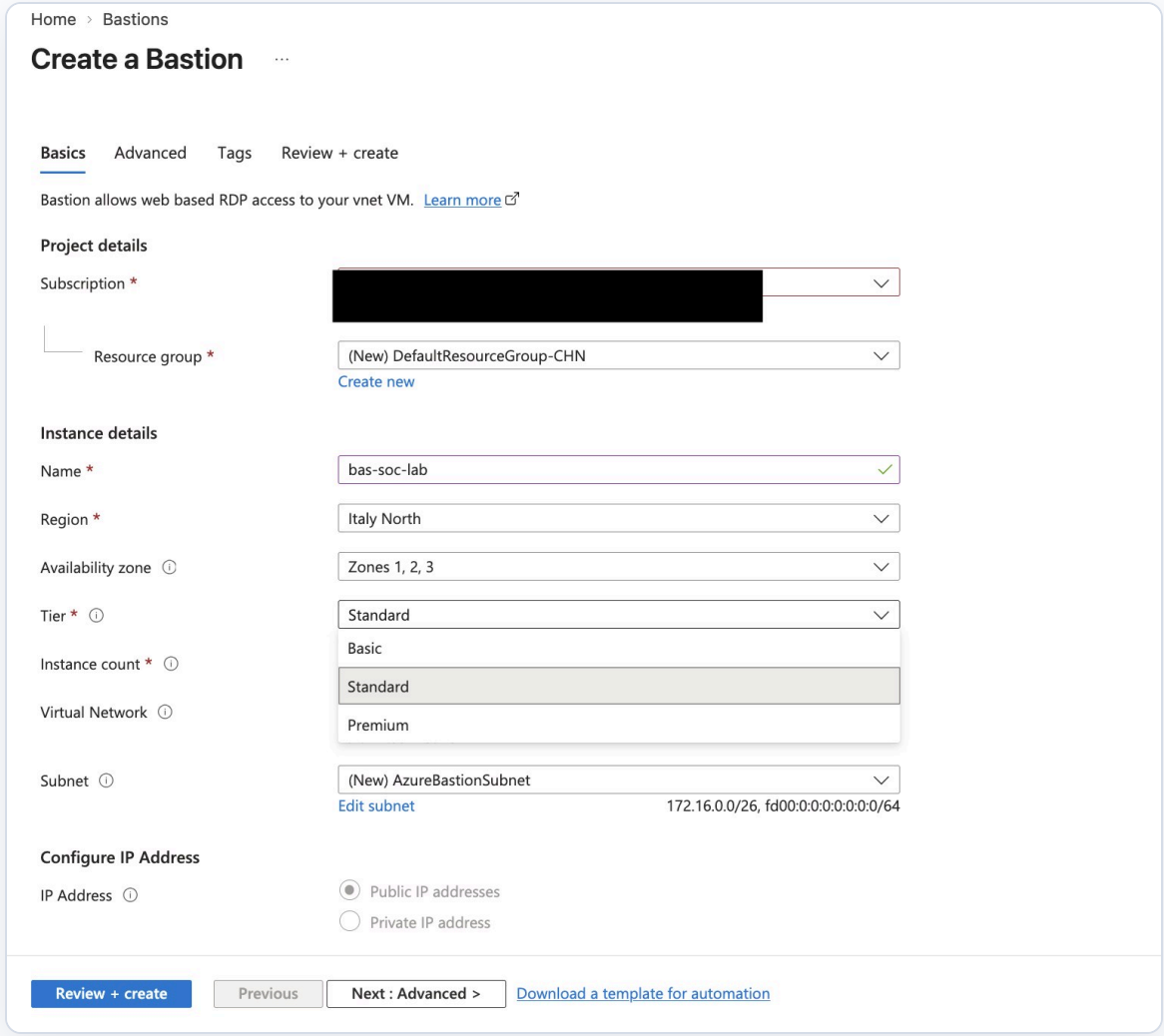
```
RG=DefaultResourceGroup-CHN
az network vnet subnet create -g $RG \
  --vnet-name vnet-openai -n AzureBastionSubnet \
  --address-prefixes 10.0.2.0/26
az network public-ip create -g $RG -n pip-bas-soc-lab \
  --sku Standard --allocation-method Static
az network bastion create -g $RG -n bas-soc-lab \
  --vnet-name vnet-openai --public-ip-address pip-bas-soc-lab \
  --sku Standard --enable-tunneling true
```

PART 10

10.1 Azure Bastion in the portal

The portal path is **Bastions, Create**. The Basics tab takes the name `bas-soc-lab`, the region and the tier; Standard is the tier this design needs, because it enables the native client and the tunnel used later.

When a virtual network is selected, the wizard proposes the subnet `AzureBastionSubnet` with a /26 range and a public address for the service, the same objects the CLI commands create. In this build the subnet is `10.0.2.0/26` in the hub.



The Bastion wizard: the proposed subnet depends on the selected network

PART 10

10.2 Enable Microsoft Entra ID login on the VMs

WHY SSH keys are copied and rarely revoked; Entra ID login applies MFA and instant revocation.

USED FOR Every administrative login from now on.

```
for vm in vm-soc-lab vm-app-lab; do
  az vm extension set -g $RG --vm-name $vm \
    --publisher Microsoft.Azure.ActiveDirectory \
    --name AADSSHLoginForLinux
done
```

The loop installs the Entra ID login extension on both machines. The VMs reach Entra ID to validate certificates and roles through the NAT Gateway. With Entra ID login, access follows the company identity: MFA applies to every VM sign-in, and access is decided by Azure roles at the moment of each login.

`grp-vm-admins` holds Virtual Machine Administrator Login on the resource group. Now that both VMs and Bastion exist, it also receives **Reader** on exactly those resources, which Bastion needs to open a session.

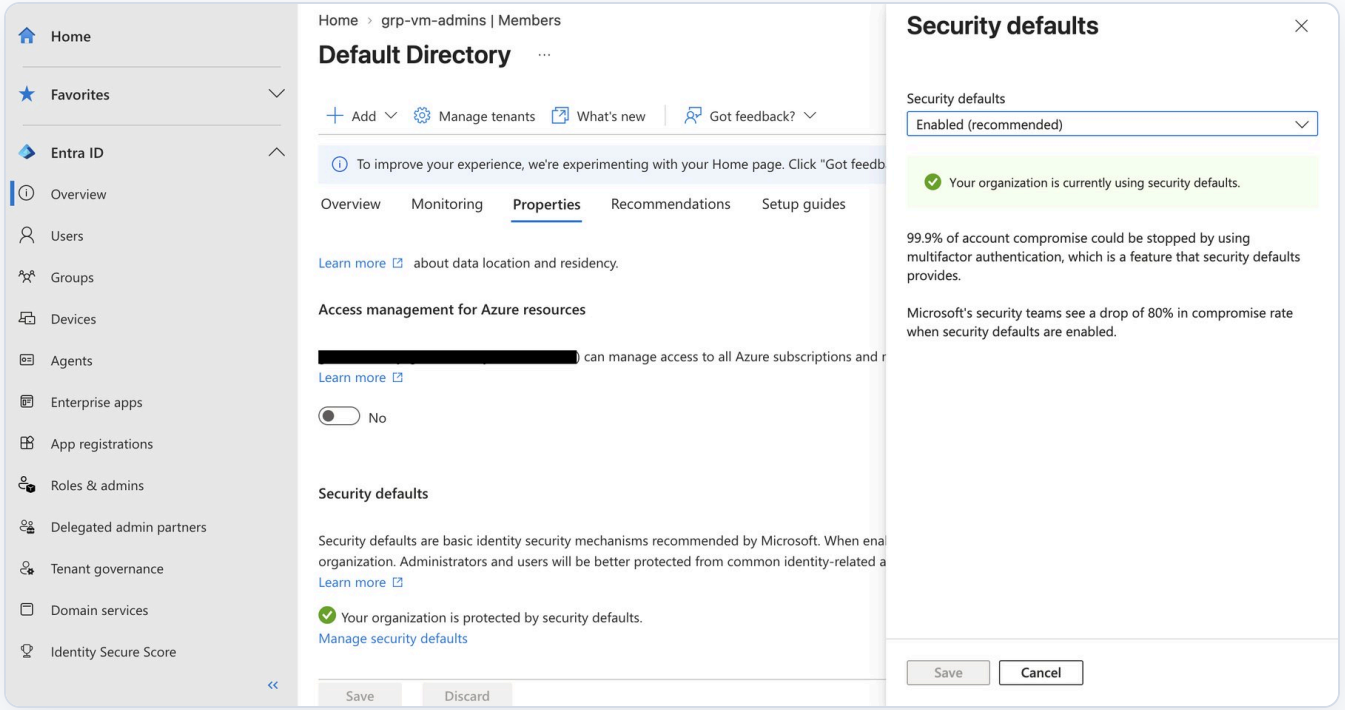
```
GID=$(az ad group show -g grp-vm-admins --query id -o tsv)
BAS=$(az network bastion show -g $RG -n bas-soc-lab --query id -o tsv)
VMS=$(az vm list -g $RG \
  --query "[].[id, networkProfile.networkInterfaces[0].id]" -o tsv)
for id in $VMS $BAS; do     # each VM, its NIC, and Bastion
  az role assignment create --assignee-object-id "$GID" \
    --assignee-principal-type Group --role Reader --scope "$id"
done
```

PART 10

10.2 MFA for every sign-in: security defaults

Entra ID login is only as strong as the sign-in behind it. In a tenant without Conditional Access policies, Microsoft Entra ID **security defaults** provide the free baseline: every user must register for multifactor authentication, administrators are asked for MFA when they sign in, and legacy authentication protocols, which cannot perform MFA, are blocked.

The setting lives in Entra ID, Overview, Properties, Manage security defaults, and here it reads **Enabled**. Organizations with Entra ID P1 replace security defaults with Conditional Access policies, for example requiring both MFA and a compliant device for administrative access.



Security defaults enabled

PART 10

10.3 Remove the public IP, the SSH rule and the SSH keys

WHY The last public entry point and the last personal credentials: removing them leaves Entra ID via Bastion as the only way in.

USED FOR The final posture: nothing reachable from the internet.

```
NIC=$(az vm show -g $RG -n vm-soc-lab --query "networkProfile.networkInterfaces[0].id" -o tsv)
IPCFG=$(az network nic show --ids "$NIC" --query "ipConfigurations[0].name" -o tsv)
# Detach and delete the public IP
az network nic ip-config update --ids "$NIC/ipConfigurations/$IPCFG" --remove publicIpAddress
az network public-ip delete -g $RG -n vm-soc-lab-ip
# Delete the inbound SSH rule
az network nsg rule delete -g $RG --nsg-name vm-soc-lab-nsg -n SSH
# Remove the build-phase key and the internal key from the jump host
az vm run-command invoke -g $RG -n vm-soc-lab --command-id RunShellScript --scripts \
  "truncate -s 0 /home/azureuser/.ssh/authorized_keys; rm -f /home/azureuser/.ssh/id_app*"
# Remove the internal key from the target
az vm run-command invoke -g $RG -n vm-app-lab --command-id RunShellScript --scripts \
  "sed -i '/azureuser@vm-soc-lab/d' /home/azureuser/.ssh/authorized_keys"
```

The public IP is detached and deleted, since a detached address would still be billed, and the SSH rule disappears. Bastion reaches the jump host through the default rule for virtual network traffic and the target through its explicit Bastion rule of step 7.2. Run-command executes each script through the Azure VM agent, which works even without SSH. From this point neither VM has a public IP, no NSG has an inbound rule from the internet, outbound traffic leaves only through the NAT Gateway, and every action on the workload is tied to a person rather than to the shared `azureuser` account.

PART 10

10.4 Connect through Bastion and validate

WHY Confirms administration works with no public IP and no SSH key.

USED FOR Daily operations.

In the portal, an administrator opens `vm-soc-lab`, chooses **Connect via Bastion** and selects **Microsoft Entra ID** as the authentication type. From a terminal, the Azure CLI opens the same session with the native client of the Standard tier.

The portal of the application is reached with a Bastion tunnel, which opens port 8080 on your computer and forwards it through Bastion to port 80 of the target, like the SSH tunnel of Part 7 but authenticated with Entra ID and Azure roles instead of a key.

The change is validated when `whoami` on the VM returns your Entra ID user principal name, when neither VM shows a public IP or an inbound internet rule, and when the old private key no longer opens a session.

```
az login
az extension add -n bastion
az extension add -n ssh
RG=DefaultResourceGroup-CHN
VM_ID=$(az vm show -g $RG -n vm-soc-lab --query id -o tsv)
az network bastion ssh -n bas-soc-lab -g $RG \
  --target-resource-id "$VM_ID" --auth-type AAD

APP_ID=$(az vm show -g $RG -n vm-app-lab --query id -o tsv)
az network bastion tunnel -n bas-soc-lab -g $RG \
  --target-resource-id "$APP_ID" \
  --resource-port 80 --port 8080
```

10.5 Alternative: Just-In-Time VM access

Some machines cannot avoid a public IP. For those, **Just-In-Time VM access** in Microsoft Defender for Cloud, part of Defender for Servers Plan 2, is the alternative to Bastion.

The management ports of the VM stay closed by default behind a deny rule that Defender for Cloud manages in the network security group. When an authorized user needs to connect, they request access, and Defender for Cloud opens port 22 only for that user's source address and only for a limited window, for example three hours.

When the window ends the port closes again on its own, and every request is recorded in the Azure Activity log for later review.

Bastion with Entra ID login removes public exposure completely, which is why it is the model used in this design. Just-In-Time access is the right control when a public IP truly cannot be avoided, because it reduces the exposure to the few hours in which someone is actually working.

WRAP-UP

Security controls summary

Governance comes first: Azure Policy restricts the region, gives every resource an owner, a cost center and an environment tag, and denies any AI service with open network access or API keys.

Network. Azure OpenAI has public access disabled and is reachable only through its private endpoint, and the test from outside proves it. The hub-and-spoke topology and the private DNS zone keep name resolution private. No VM has a public IP, the target accepts traffic only from the jump host and Bastion, outbound traffic leaves only through the NAT Gateway, and administrative access enters only through Azure Bastion over TLS.

Identity. Key-based authentication is disabled on Azure OpenAI, workloads use managed identities, and no secret is stored anywhere. People receive least-privilege roles through Entra ID groups, sign in to the VMs with Entra ID behind MFA, and both the build-phase key and the internal key have been removed.

Application. The workload follows secure coding practices, with salted password hashes, parameterized SQL, constant-time sign-in checks, security headers, an unprivileged service account and a secret kept outside the code.

Monitoring rests on structured security events, a syslog baseline with a severity threshold for each facility, the audit logs of Azure OpenAI and a Log Analytics workspace with Microsoft Sentinel on top. The model itself returns only text, cannot act on anything, and its output is always reviewed by the analyst.

Costs are bounded by the daily ingestion cap, a defined retention period, budget alerts and a single resource group that can be removed in one operation. The gaps found in the post-build review are listed on slide 98.

WRAP-UP

Cost management and teardown

Every component is billed differently. Sentinel and Log Analytics charge per gigabyte ingested, with a free trial in the first thirty-one days. Virtual machines are billed per hour while they run, so they are deallocated when idle. Azure Bastion Standard is billed per hour and is one of the largest fixed costs, so it is worth deleting when nobody needs it. The NAT Gateway is billed per hour plus the data it processes, and Azure OpenAI charges per token, which for a few assessments is very little.

Deallocating releases the compute, which a shutdown from inside the operating system does not do. When the work is finished, one command deletes the resource group with everything in it. The four Entra ID groups and the soft-deleted Azure OpenAI resource live outside it: delete the groups and purge the resource to release its name. If the workspace is connected to the Defender portal, disconnect it first.

```
RG=DefaultResourceGroup-CHN
# pause: deallocate the VMs
az vm deallocate -g $RG -n vm-soc-lab --no-wait
az vm deallocate -g $RG -n vm-app-lab --no-wait

# teardown: the whole lab
az group delete --name $RG --yes
```

WRAP-UP

Known limitations and what I would change

The gaps found when the lab was reviewed after the build, and the fix for each.

In this lab	What I would change
Policies were assigned after the first resources, so early screenshots show no tags	Assign policy before any resource and remediate what already exists (slide 16)
Policies scoped to one resource group	Assign them at management group level, so every subscription inherits them (slide 15)
VM administrators held Reader on the whole resource group, workspace included	Reader only on the VMs, NICs and Bastion; workspace in its own group (slides 60, 62, 91)
During the build phase, the jump host reached the target with a shared, passphrase-less key	Bastion with Entra ID to every VM; the internal key is removed (slide 93)
The attack ran from the jump host, so attacker and administrator share an address	Separate attacker machine; automatic blocks exclude admin and Bastion ranges (slide 83)
Usernames were logged unescaped, so the parsed source IP could be forged	JSON events parsed with parse_json(), filtered on ProcessName only (slides 71, 84)
Azure Activity, Entra ID and Bastion logs were not sent to the workspace	Send them, and alert when the AI resource's network or key settings change
nginx's syslog tag was parsed as the host name; the VMs had pending updates	Fix the nginx syslog format; enable patch orchestration (slides 45, 82)
Everything was built with the portal and the Azure CLI	Bicep or Terraform, with policies and detections managed as code

From lab to enterprise · governance, identity, network

This project already applies the core enterprise controls, and the same design grows naturally when it moves to production. **Governance** moves up to an Azure landing zone, with management groups and separate subscriptions for connectivity, identity, management and workloads as described by the Cloud Adoption Framework, and policies are assigned at management group level with Deny effects and complete initiatives such as the Microsoft Cloud Security Benchmark.

Identity is hardened with Privileged Identity Management, so administrative roles are eligible rather than permanent and are activated on demand with a justification, MFA and, where needed, an approval. Conditional Access requires compliant devices and MFA for the management plane and for VM sign-in, group memberships are reviewed periodically, and break-glass accounts are kept outside Conditional Access but closely monitored.

On the **network**, an Azure Monitor Private Link Scope lets the agents send their data to Log Analytics through private endpoints instead of public service endpoints, so that even telemetry never touches a public address.

An Azure DNS Private Resolver in the hub allows on-premises networks connected through VPN or ExpressRoute to resolve the private zones, and the private DNS zones are centralized in the connectivity subscription and linked to every spoke, so every workload resolves every private endpoint the same way. Flow logs and DDoS protection complete the picture on the hub.

From lab to enterprise · AI, security operations, workload

Around the **AI service**, an Azure API Management gateway in front of Azure OpenAI centralizes authentication, rate limiting, token quotas per team, request logging and failover between deployments. Content filters and quotas are defined per use case, and because security logs contain attacker-controlled text, model output is always treated as untrusted and never triggers an action without human or rule-based validation.

Data protection becomes a formal step of the go-live: usernames and IP addresses in logs are personal data under GDPR, so a data protection impact assessment, a deliberate data residency choice such as an EU Data Zone deployment and an AI usage policy are part of the project.

One gap found in this lab comes first: the Azure Activity, Entra ID and Bastion logs in the workspace, with an alert when the AI resource's network or key settings change.

Security operations widen their visibility with Entra ID sign-in and audit logs, Azure Activity, Microsoft Defender XDR and diagnostic settings for every resource, deployed automatically through policy, while Defender for Cloud adds posture management, endpoint protection and vulnerability assessment for the servers, and threat protection for AI workloads. Retention follows the logging policy and regulations such as NIS2 and ISO 27001, the health of the pipeline is monitored with alerts on missing heartbeats and ingestion drops, and detections are managed as code.

The **workload** gains a web application firewall and TLS with certificates in Key Vault, a hardening baseline, managed patching, backup and a disaster recovery plan, and the whole environment is described as infrastructure as code, reviewed and deployed through CI/CD.

DESIGN ONLY

Design: the automation part

The foundation is designed so that the next step plugs straight into it. It starts with **detection**: a scheduled analytics rule in Microsoft Sentinel watches the application events and raises an alert when five or more sign-ins fail from the same address within ten minutes, mapped to MITRE ATT&CK technique T1110. The simulated attack of Part 9 then becomes an incident, carrying the attacker's address, the targeted accounts and the host as entities.

Next comes **AI triage**. A playbook on Logic Apps Standard, integrated with the virtual network and running with its own managed identity that holds Cognitive Services OpenAI User, takes the incident and its log lines, sends them to the private model through the private endpoint, and writes the assessment back onto the incident. A Consumption-plan playbook could not reach a model that has no public path.

The analyst then receives that assessment and decides whether to **block** the attacker or dismiss the incident. On a block decision, a deny rule for the attacker's address is added to the network security group of the workload subnet, never covering the administrator or Bastion ranges; it removes itself automatically after a set period, and every action is recorded on the incident.

The principle stays the same throughout: the AI recommends, a human approves and the automation acts, and every action is narrow, temporary and audited.

THANKS FOR READING

Private by design. Validated at every step.

The code and the KQL queries are in the repository.

Known limitations: slide 98 · Automation design: slide 101